



**MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA**

**Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering**

VREMERE ADRIAN, FAF-232

Report

Laboratory Work n.5.2

Automatic Temperature Regulation with PID Control

on Embedded Systems

Checked by:

Martîniuc Alexei, *univ. asist.*
FCIM, UTM

Chişinău – 2026

1. Domain Analysis

1.1. Objective of the Laboratory Work

The objective of this laboratory work is to implement an embedded PID control system for a physical process. The selected application is temperature regulation by forced ventilation: a DHT11 sensor measures the ambient temperature, the user selects the desired temperature setpoint, and a DC fan driven through an H-bridge receives a PWM command computed by a proportional-integral-derivative controller.

The work is prepared primarily for a real Arduino Mega 2560 setup. Wokwi is kept as a reproducible wiring and firmware reference for the supported logic-side components, but the L293D/L298-style motor driver, external 9 V supply, and fan branch must be validated on the physical circuit. This choice follows the provided hardware image, where the fan is powered separately from the Arduino logic rail and receives only control signals from the MCU.

1.2. Problem Definition

“Să se proiecteze și să se implementeze o aplicație modulară pentru microcontroler (MCU) care realizează controlul PID asupra unui parametru fizic. Sistemul va achiziționa periodic valoarea parametrului de la un senzor adecvat, va compara cu valoarea de referință configurabilă și va calcula acțiunea de control utilizând algoritmul PID. Parametrii PID vor fi configurabili din cod sau din interfață, iar valorile parametrului măsurat, ale Set Point-ului, ale semnalului de control și starea actuatorului vor fi afișate pe LCD și/sau prin interfața serială.”

The implemented system uses the DHT11 temperature as the measured process value and the fan speed as the manipulated output. Because the fan cools the environment, the controller is configured in reverse action: the error used by the PID algorithm increases when the measured temperature rises above the setpoint. The resulting control signal is limited to the range 0–100%, then converted to a PWM duty cycle for the fan driver.

The setpoint is configurable through two mechanisms. The potentiometer on A0 provides the default continuous setpoint over the 18–35°C range. The 4x4 keypad can switch to manual mode, increment or decrement the manual setpoint, or confirm a numeric setpoint entry. The PID coefficients are configured in code and can also be changed at runtime through keypad-selected presets named SOFT, NORM, and FAST.

1.3. Technical Requirements and Validation Criteria

ID	Requirement	Validation Criterion
R1	Acquire temperature and humidity from the DHT11 without blocking unrelated application logic.	LCD and serial channels update after each acquisition cycle; invalid samples set 'Valid:0'.
R2	Provide a configurable temperature set-point through potentiometer and keypad input.	A0 changes 'SetPoint'; keypad manual entry overrides the potentiometer after confirmation.
R3	Compute a bounded PID output from set-point, measured value, and configured gains.	Serial Plotter shows 'Output', 'Error', 'Kp', 'Ki', and 'Kd'; output remains between 0 and 100%.
R4	Apply the PID output to a PWM fan actuator through an H-bridge driver.	Fan duty follows 'Duty'; the motor is stopped when the sensor is invalid or the PID output is near zero.
R5	Separate acquisition, control, actuation, input, and display into FreeRTOS tasks.	Source tree contains dedicated task files and uses semaphores/mutexes for coordination.
R6	Report process values for real-time analysis.	Arduino Serial Plotter receives named fields 'SetPoint', 'Value', 'Output', 'Duty', and 'Error'.

Table 1: Technical requirements and validation criteria

1.4. Used Technologies

PID Control

The PID controller computes a continuous control signal from the current regulation error. The proportional term reacts to the current error, the integral term accumulates past error in order to reduce steady-state offset, and the derivative term reacts to the rate of change of the error. In discrete embedded software the controller is evaluated at a known sampling interval:

$$u(k) = K_p e(k) + K_i \sum e(k) \Delta t + K_d \frac{e(k) - e(k-1)}{\Delta t}$$

where $u(k)$ is the command signal, $e(k)$ is the current error, and K_p , K_i , and K_d are the configured gains.

For this cooling application the controller uses reverse action:

$$e(k) = T_{measured}(k) - T_{setpoint}(k)$$

so the fan command increases only when the measured temperature is higher than the desired value. The output is clamped to the physical range 0–100% and the integral contribution is also clamped to reduce windup when the output saturates.

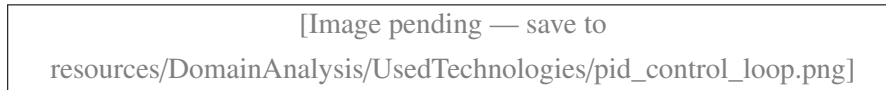


Figure 1.1 – PID closed-loop control concept

PWM Motor Control

Pulse-width modulation controls the average voltage applied to a DC motor by switching the driver rapidly between ON and OFF states. The Arduino ‘analogWrite’ function produces an 8-bit PWM command from 0 to 255, which is mapped in the firmware from a duty cycle expressed as a percentage. The motor driver receives a PWM enable signal and two direction inputs. In this lab the fan is operated in one direction only, therefore IN1 is held HIGH, IN2 is held LOW, and the enable pin receives the PID-controlled duty cycle.

I2C LCD Interface

The LCD 1602 display uses an I2C backpack, so it requires only SDA, SCL, 5 V, and GND. The same I2C interface is used in the previous laboratories and is reused here to display current temperature, setpoint, fan duty, PID error, and selected gain preset.

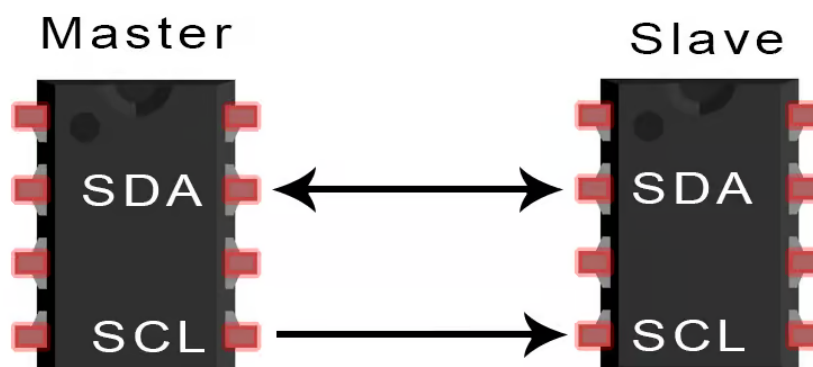


Figure 1.2 – I2C protocol used by the LCD module

FreeRTOS Task Scheduling

FreeRTOS is used to separate periodic and event-driven responsibilities. The input task scans the keypad every 50 ms, the acquisition task samples the DHT11 and potentiometer every

2000 ms, the control task waits for a new sample, the actuation task waits for a fresh control command, and the display task updates the LCD and Serial Plotter every 500 ms. A mutex protects shared state and binary semaphores express the order acquisition → control → actuation.

1.5. Hardware Components

Arduino Mega 2560

The Arduino Mega 2560 provides enough GPIO for the keypad matrix, DHT11 data line, I2C LCD, potentiometer input, and H-bridge motor driver. Its PWM-capable D3 pin is used as the fan enable command, while D16 and D17 provide the fan direction signals.

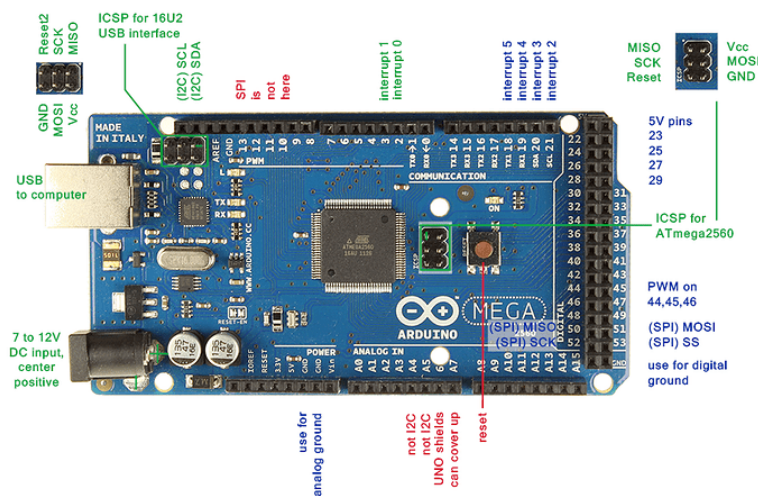


Figure 1.3 – Arduino Mega 2560 development board

DHT11 Temperature and Humidity Sensor

The DHT11 sensor measures the process temperature and reports humidity as a diagnostic value. Its data line is connected to D2. A pull-up resistor is shown in the wiring reference because the data bus must return to logic HIGH between sensor transactions.

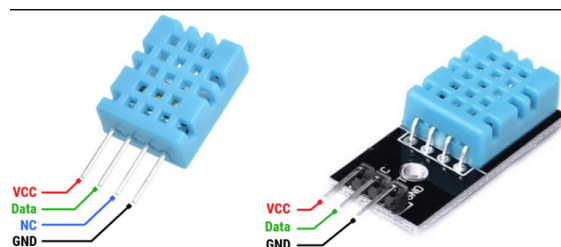


Figure 1.4 – DHT11 temperature and humidity sensor

Potentiometer

The potentiometer forms a voltage divider between 5 V and GND, and its wiper is connected to A0. The firmware maps the ADC value linearly to the allowed setpoint range, making it possible to test the controller without a computer-side command interface.

[Photo pending — save to
resources/DomainAnalysis/Hardware-
Components/potentiometer.png]

Figure 1.5 – Potentiometer used for setpoint selection

4x4 Matrix Keypad

The keypad provides local interaction. Key A toggles between potentiometer and manual setpoint mode, B and C decrease or increase the manual setpoint, D cycles the PID preset, digits enter a numeric setpoint, '#' confirms the value, and '*' cancels the edit.

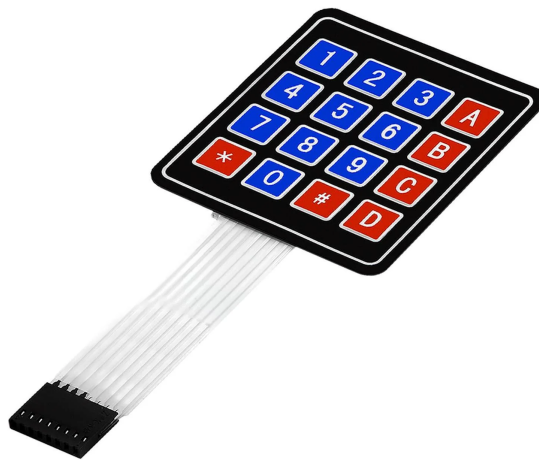


Figure 1.6 – 4x4 membrane matrix keypad

LCD 1602 I2C Display

The LCD gives direct feedback during physical testing. It alternates between process values, controller error/output, and PID gains so the board can be operated without relying only on the Serial Plotter.



Figure 1.7 – LCD 1602 module with I2C backpack

L293D/L298-Style H-Bridge Driver

The motor driver isolates the Arduino logic pins from the higher-current fan load. The enable input receives PWM from D3, while D16 and D17 select the fan direction. The physical branch uses an external 9 V supply, so the Arduino GND and driver logic GND must share a reference.

[Photo pending — save to resources/DomainAnalysis/HardwareComponents/h_bridge_driver.png]

Figure 1.8 – L293D/L298-style H-bridge motor driver

DC Fan and External Supply

The DC fan is the physical actuator. The 9 V source powers the motor side of the driver, while the Arduino only provides control logic and PWM. This separation is important because the fan current should not be drawn from the Arduino 5 V regulator.

[Photo pending — save to resources/DomainAnalysis/HardwareComponents/dc_fan.png]

Figure 1.9 – DC fan used as the PID-controlled actuator

1.6. Software Components

PlatformIO is used to build the Arduino Mega firmware, manage external libraries, and select the active lab environment. Wokwi is used as a partial wiring reference for the logic-side circuit. Full actuator validation remains a physical test because the provided fan/driver branch is not completely represented in the simulator.

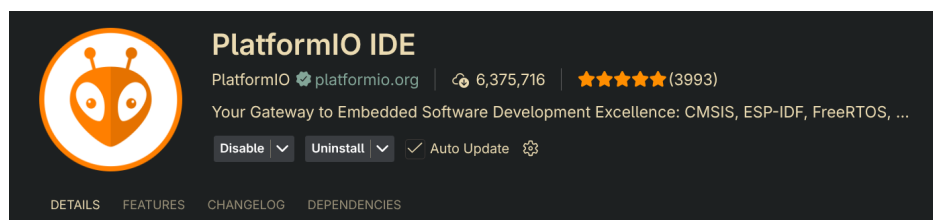


Figure 1.10 – PlatformIO development environment

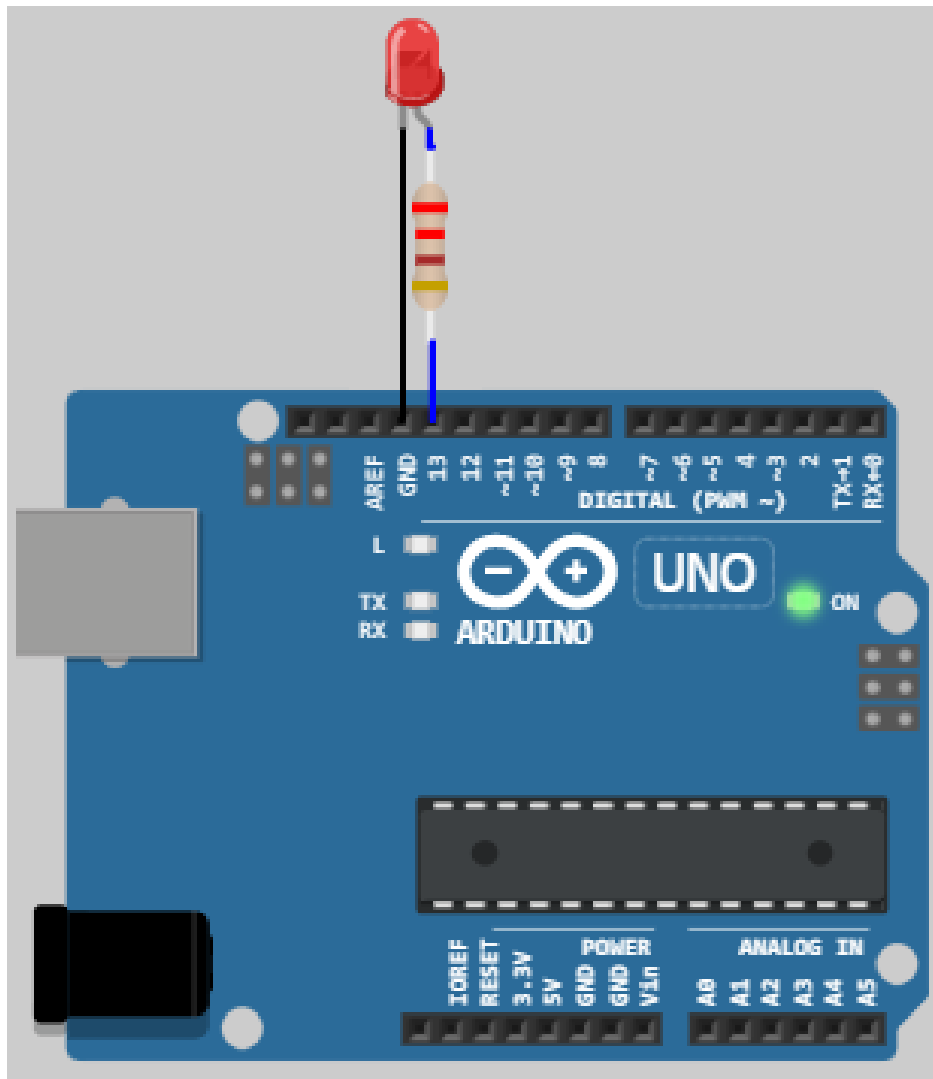


Figure 1.11 – Wokwi wiring reference environment

1.7. Application Context

PID temperature control is widely used in electronics cooling, incubators, environmental chambers, greenhouse ventilation, and process-control equipment. A fan-based controller is intentionally simple but demonstrates the same design pattern used in larger HVAC systems: measure the temperature, compare it with a setpoint, compute a continuous command, and apply the command through a power driver. The embedded implementation must also consider sensor update rate, actuator saturation, invalid measurements, and clear diagnostic reporting.

2. Design

2.1. System Architecture Diagrams

Component-Level Architecture

The system is organized as a closed feedback loop. The DHT11 sensor provides the measured temperature, the potentiometer and keypad provide the desired setpoint and tuning choices,

the PID controller computes a continuous command, and the H-bridge applies the command to the fan through PWM. The LCD and Serial Plotter expose the same internal state so that the physical behavior can be checked while the board is running.

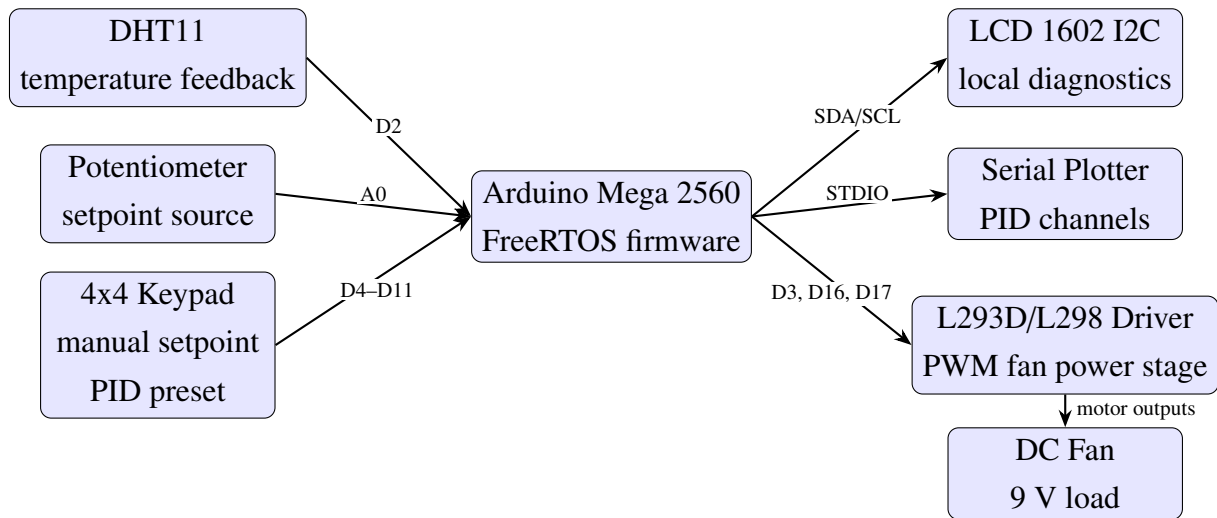


Figure 2.1 – System structural diagram for PID temperature regulation

Closed-Loop Control Structure

The structural diagram shows the hardware connections, but the control loop itself is better understood as a signal chain. The setpoint and measured temperature meet at the error calculation, the PID controller converts the error into a normalized command, and the actuator conditioning block translates the command into a fan duty cycle that can actually be applied to the motor. The DHT11 closes the loop by measuring the temperature after the fan has affected the air around the sensor.

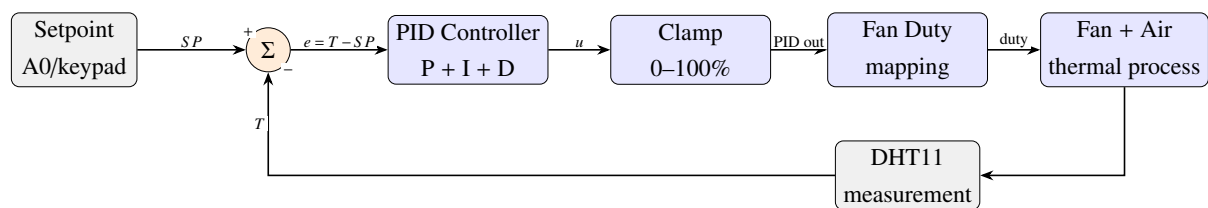


Figure 2.2 – Closed-loop PID signal structure

Layered System Architecture

The layered architecture keeps the control algorithm separate from hardware access. The application layer creates tasks and defines the runtime sequence. The service layer contains shared state and the PID controller. The ECAL layer contains reusable wrappers for DHT11, analog setpoint input, LCD, keypad, PWM, and H-bridge motor control. The hardware layer contains the actual Arduino pins and external driver circuit.

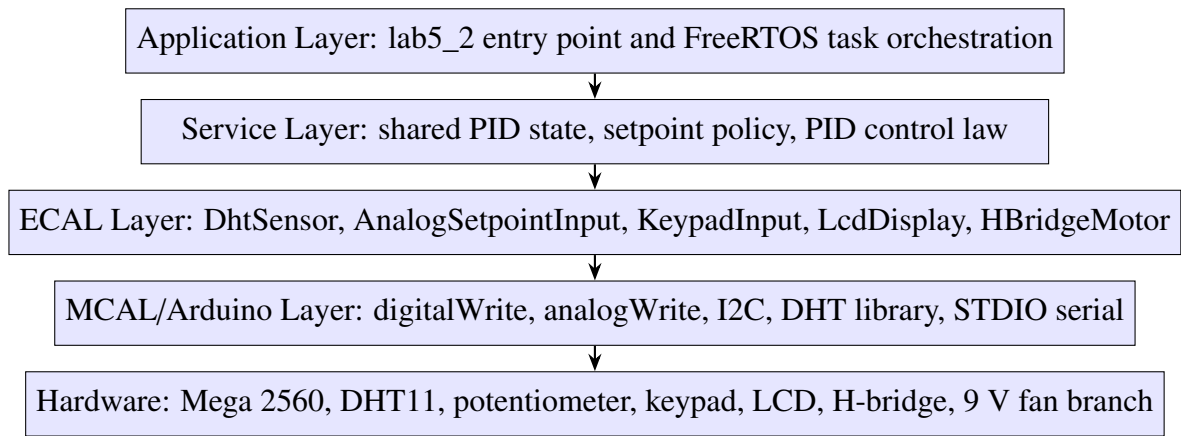


Figure 2.3 – Layered architecture of the PID control application

2.2. Block Diagrams

PID Control Flow

The control task runs only after the acquisition task publishes a fresh sample. The PID controller is updated outside the shared-state critical section, so the mutex is held only while copying input values and writing the computed result.

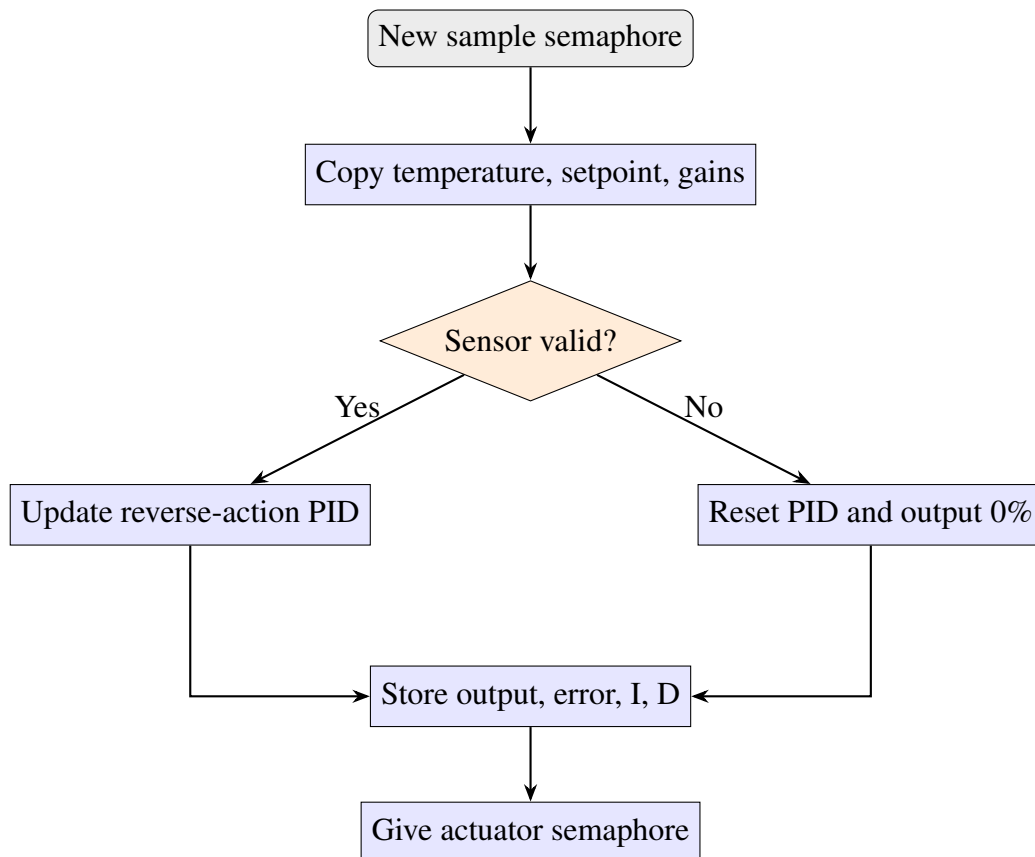


Figure 2.4 – Flowchart of the PID control task

PID Term Decomposition

The PID controller is implemented as three explicit contributions. The proportional part reacts immediately to the current error, the integral part stores accumulated error over time, and the derivative part estimates the error trend from the previous sample. Keeping these terms visible in the design is useful during tuning because an unstable response can usually be traced to one dominant term: excessive proportional gain causes aggressive reaction, excessive integral gain causes slow windup, and excessive derivative gain amplifies noisy measurements.

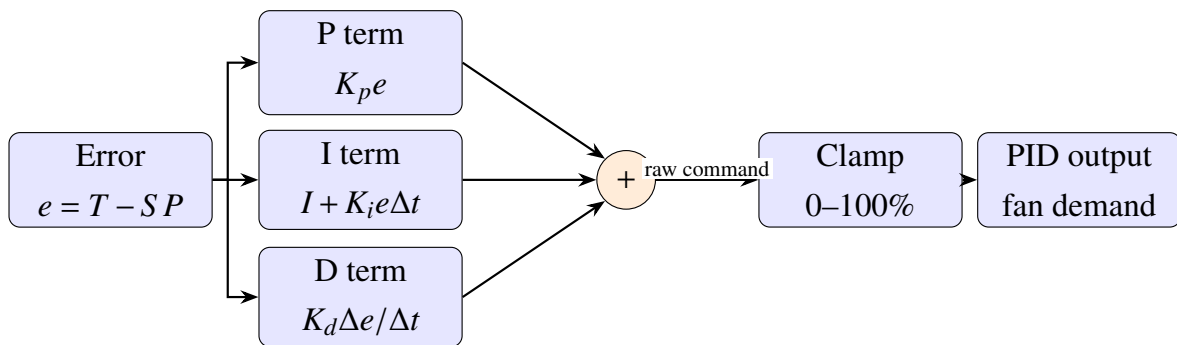


Figure 2.5 – PID proportional, integral, and derivative term decomposition

FreeRTOS Data Flow

The shared state is the only communication contract between tasks. The acquisition task produces process measurements, the input task produces user settings, the control task produces a bounded control output, and the actuation task applies the PWM command to the fan driver.

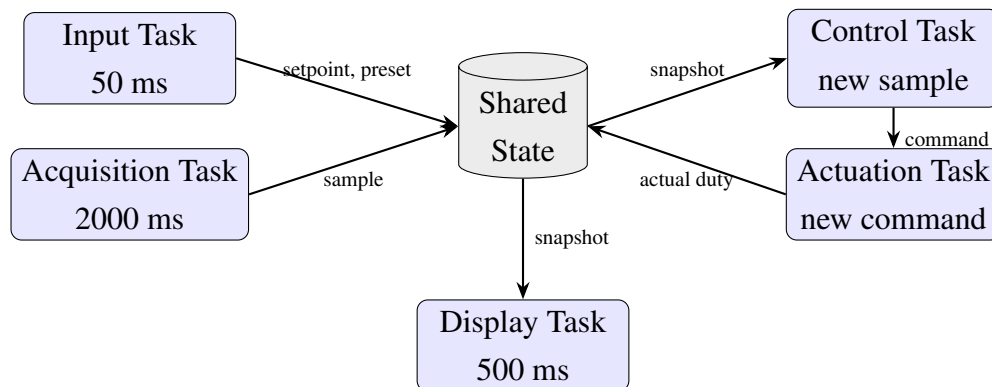


Figure 2.6 – FreeRTOS task data flow

Synchronization Sequence

The control chain is intentionally event-driven. The acquisition task is the only task that gives the new-sample semaphore. The control task blocks until this happens, computes exactly one PID output for the fresh sample, and then wakes the actuation task. This prevents the fan command from being recalculated repeatedly from stale sensor data and makes the timing relationship easy to verify in the report and on the Serial Plotter.

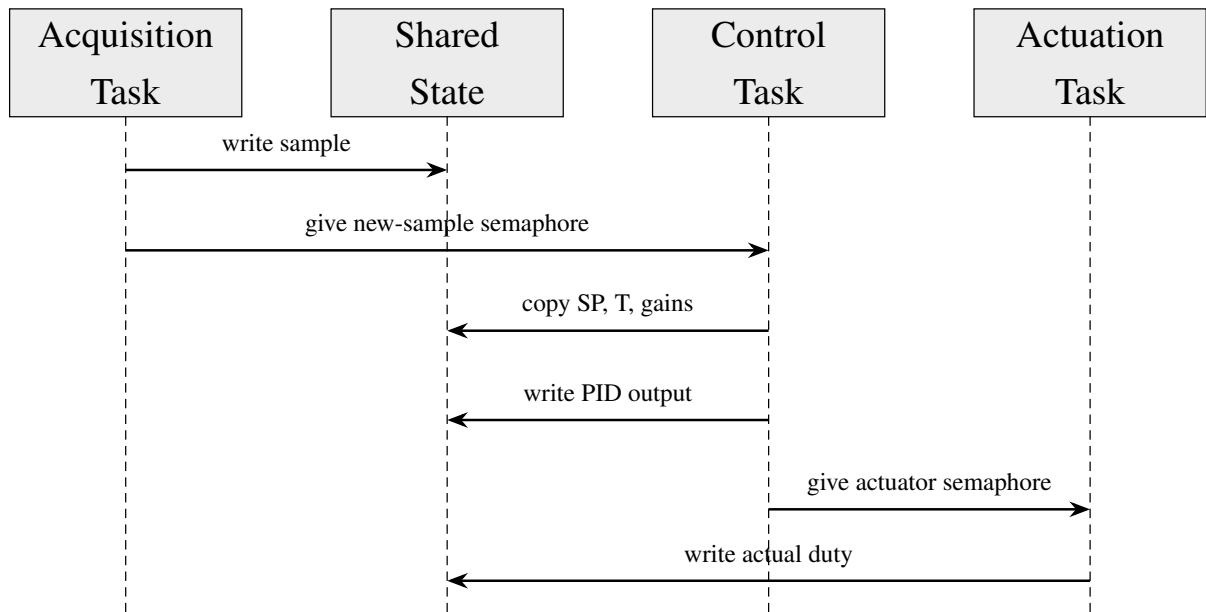


Figure 2.7 – FreeRTOS synchronization sequence for the PID control chain

Keypad Interaction States

The keypad has two major operating states: normal command mode and numeric setpoint editing. This prevents accidental numeric input from immediately changing the setpoint before confirmation.

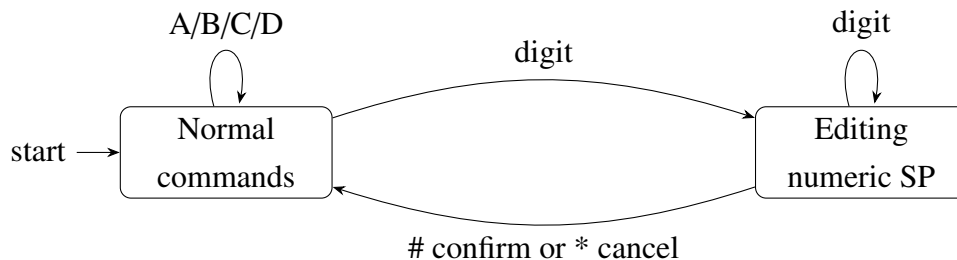


Figure 2.8 – Keypad input state diagram

2.3. Electrical Schematics

Circuit Schematic

The physical circuit extends the Wokwi-supported logic-side wiring with an H-bridge fan branch. D3 is the PWM enable signal, D16 and D17 select direction, and the external 9 V source powers the fan side of the driver. The Arduino and driver grounds must be connected so the logic signals have a common reference.

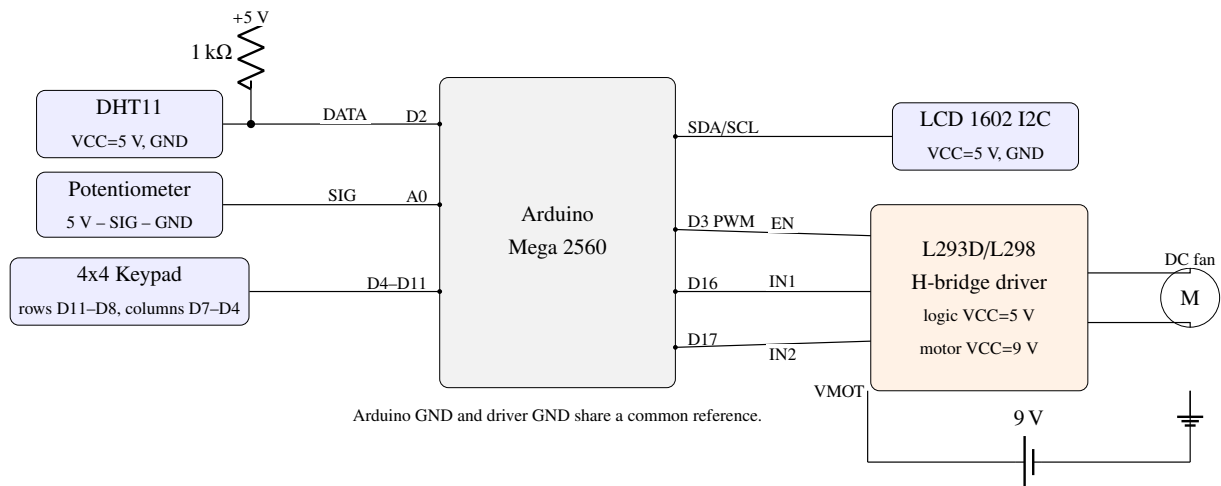


Figure 2.9 – Electrical schematic of the PID temperature control circuit

Component Specification

- Arduino Mega 2560: ATmega2560, 5 V logic, 16 MHz.
- DHT11: digital temperature and humidity sensor connected to D2.
- Potentiometer: analog setpoint source connected to A0.
- Keypad: 4x4 matrix, rows on D11–D8 and columns on D7–D4.
- LCD 1602 I2C: address 0x27, connected through SDA/SCL.
- H-bridge driver: PWM enable on D3, IN1 on D16, IN2 on D17.
- Fan branch: external 9 V supply and DC fan connected to the driver motor outputs.

Wokwi Reference

The Wokwi project contains the Arduino Mega, keypad, LCD, potentiometer, and a direct PWM motor/LED proxy. Wokwi’s available DHT-family visual component is kept only as a wiring placeholder; the physical reference schema and firmware use a DHT11 on D2. The complete L293D and 9 V fan branch are documented in the schematic and must be validated on the real board.

[Screenshot pending — start labs/wokwi/lab5.2/wokwi.toml and save the running circuit to
resources/Design/ElectricalSchematics/wokwi_circuit.png]

Figure 2.10 – Wokwi wiring reference for the supported Lab 5.2 components

Listing 2.1 – Essential Lab 5.2 pin map

{

```

"dht11": { "data": "D2", "vcc": "5V", "gnd": "GND" },
"potentiometer": { "sig": "A0", "vcc": "5V", "gnd": "GND" },
"keypad": {
  "rows": ["D11", "D10", "D9", "D8"],
  "cols": ["D7", "D6", "D5", "D4"]
},
"lcd_i2c": { "sda": "SDA", "scl": "SCL", "address": "0x27" },
"fan_driver": { "pwm": "D3", "in1": "D16", "in2": "D17" }
}

```

2.4. Modular Implementation

Project Structure

Listing 2.2 – Lab 5.2 source tree

```

labs/
|-- src/main.cpp
|-- lab/lab5_2/
|   |-- lab5_2_main.h/.cpp
|   |-- lab5_2_config.h/.cpp
|   |-- shared_state.h/.cpp
|   |-- task_input.h/.cpp
|   |-- task_acquisition.h/.cpp
|   |-- task_control.h/.cpp
|   |-- task_actuation.h/.cpp
|   '-- task_display.h/.cpp
|-- lib/
|   |-- PidController/
|   |-- HBridgeMotor/
|   |-- DhtSensor/
|   |-- AnalogSetpointInput/
|   |-- KeypadInput/
|   |-- LcdDisplay/
|   '-- StdioSerial/
'-- wokwi/lab5.2/
    |-- diagram.json
    '-- wokwi.toml

```

Module Responsibility Map

The implementation is split by responsibility rather than by peripheral alone. The task files describe when work happens, the shared-state module describes what data crosses task boundaries, and the reusable libraries describe how each hardware or control service is accessed. This organization keeps the PID calculation testable because it does not call `analogWrite`, `digitalWrite`, LCD methods, or DHT methods directly.

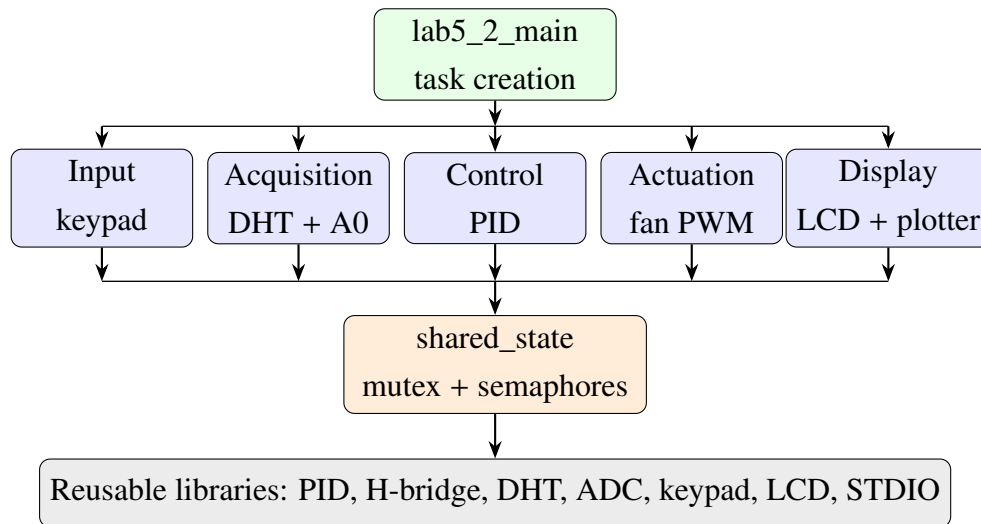


Figure 2.11 – Module responsibility map for the Lab 5.2 implementation

Application Entry Point

The lab entry point initializes STDIO serial output, prints the physical pin map and keypad commands, initializes the shared state, and creates the five FreeRTOS tasks. It also checks the return value of each `xTaskCreate` call, so stack or memory problems are visible in the serial log during hardware bring-up. The Arduino `loop()` delegate remains empty because FreeRTOS tasks own the runtime behavior after setup.

This entry point deliberately contains no PID arithmetic and no direct peripheral access. Its job is orchestration: set up the application context, create tasks with priorities and stack sizes from `lab5_2_config.h`, and leave the periodic behavior to the scheduler. That keeps the active-lab selector in `src/main.cpp` simple and preserves the repository convention that each lab is isolated behind its own setup and loop functions.

Listing 2.3 – Lab 5.2 task creation

```

lab5PidStateInit();
xTaskCreate(vTaskLab5PidInput, "Input", TASK_INPUT_STACK,
            NULL, TASK_INPUT_PRIORITY, NULL);
xTaskCreate(vTaskLab5PidAcquisition, "Acquire", TASK_ACQUISITION_STACK,
            NULL, TASK_ACQUISITION_PRIORITY, NULL);
xTaskCreate(vTaskLab5PidControl, "Control", TASK_CONTROL_STACK,
            NULL, TASK_CONTROL_PRIORITY, NULL);
xTaskCreate(vTaskLab5PidActuation, "Actuate", TASK_ACTUATION_STACK,
            NULL, TASK_ACTUATION_PRIORITY, NULL);
xTaskCreate(vTaskLab5PidDisplay, "Display", TASK_DISPLAY_STACK,
            NULL, TASK_DISPLAY_PRIORITY, NULL);
  
```

Shared State and Synchronization

The shared state stores the measured temperature and humidity, sensor validity, potentiometer setpoint, manual setpoint, active setpoint source, PID gains, PID output, applied fan duty, and

diagnostic counters. The mutex protects multi-field updates, while the new-sample and actuator semaphores enforce the control chain order.

The state is copied by value in the display task before formatting LCD and serial strings. That choice matters because LCD and serial output are relatively slow compared with memory access; keeping those operations outside the mutex prevents display refreshes from delaying acquisition or control. The control task also copies only the fields it needs, releases the mutex, runs the PID calculation, and then reacquires the mutex to publish the result. This keeps the critical section short and predictable.

Listing 2.4 – Shared PID fields

```
struct Lab5PidState {  
    float measuredTempC;  
    float activeSetpointC;  
    float kp, ki, kd;  
    float errorC;  
    float controlOutputPercent;  
    float appliedDutyPercent;  
    bool sensorValid;  
    bool fanRunning;  
};
```

Input and Acquisition Tasks

The input task owns all keypad interpretation. Key A toggles between potentiometer and manual setpoint mode, keys B and C adjust the manual setpoint in 0.5°C steps, key D cycles the PID gain preset, numeric keys fill a small edit buffer, # confirms the typed setpoint, and * cancels the edit. The keypad wrapper handles debounce, while the task itself updates only user-intent fields in shared state. It does not calculate PID output, actuate the fan, or write to the display.

The acquisition task owns the sensor and analog setpoint reads. It performs DHT11 and ADC operations before locking the shared state so slow I/O does not hold the mutex. After the sample is written, it gives the new-sample semaphore. This event is the only trigger for the PID control task, which means the controller runs at the real sensor update rate instead of spinning in a tight loop.

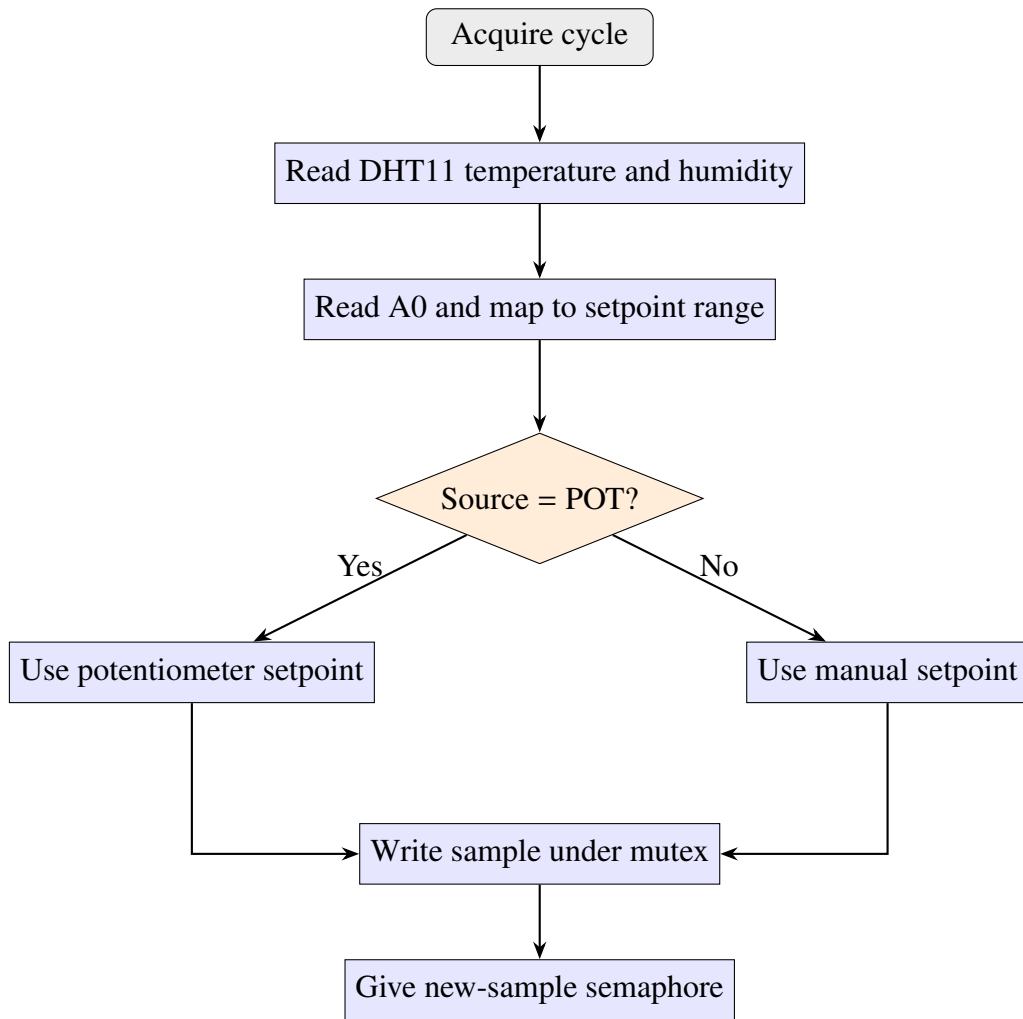


Figure 2.12 – Flowchart of acquisition and active setpoint selection

PID Controller Module

The reusable `PidController` accepts gains, output limits, and a direction mode. In reverse mode it calculates error as measured value minus setpoint, which is appropriate for a cooling fan. The integral term is clamped to the output limits to reduce windup when the actuator saturates.

Every call to `update` receives the measured value, the setpoint, and the elapsed time in seconds. The elapsed time is derived from FreeRTOS ticks, with the 2000 ms acquisition period used as a fallback for the first cycle. The controller stores the previous error internally, so the derivative term is not active until at least two samples have been seen. When the DHT11 sample is invalid, the control task resets the PID state and publishes a zero output; this prevents stale integral memory from restarting the fan at a nonzero duty after a sensor fault.

Listing 2.5 – PID update interface

```

float output = s_pid.update(setpoint, temperature, dtSeconds);
state->controlOutputPercent = output;
state->errorC = s_pid.getError();
state->pidIntegral = s_pid.getIntegral();
state->pidDerivative = s_pid.getDerivative();
  
```

H-Bridge Fan Driver

The `HBridgeMotor` wrapper combines a PWM actuator with two direction pins. The fan is commanded forward only, but the wrapper is bidirectional so the same module can be reused for rotor-position or motor-speed variants.

The actuation task does not apply the PID output directly as raw PWM. A small conditioning step first maps the 0–100% PID demand into a physical duty cycle. Very low PID values are treated as stop commands, and nonzero commands are lifted to a minimum running duty so the real fan can overcome static friction. This distinction is important on physical hardware because a motor may not start at 5% or 10% duty even though the PWM signal is technically present.

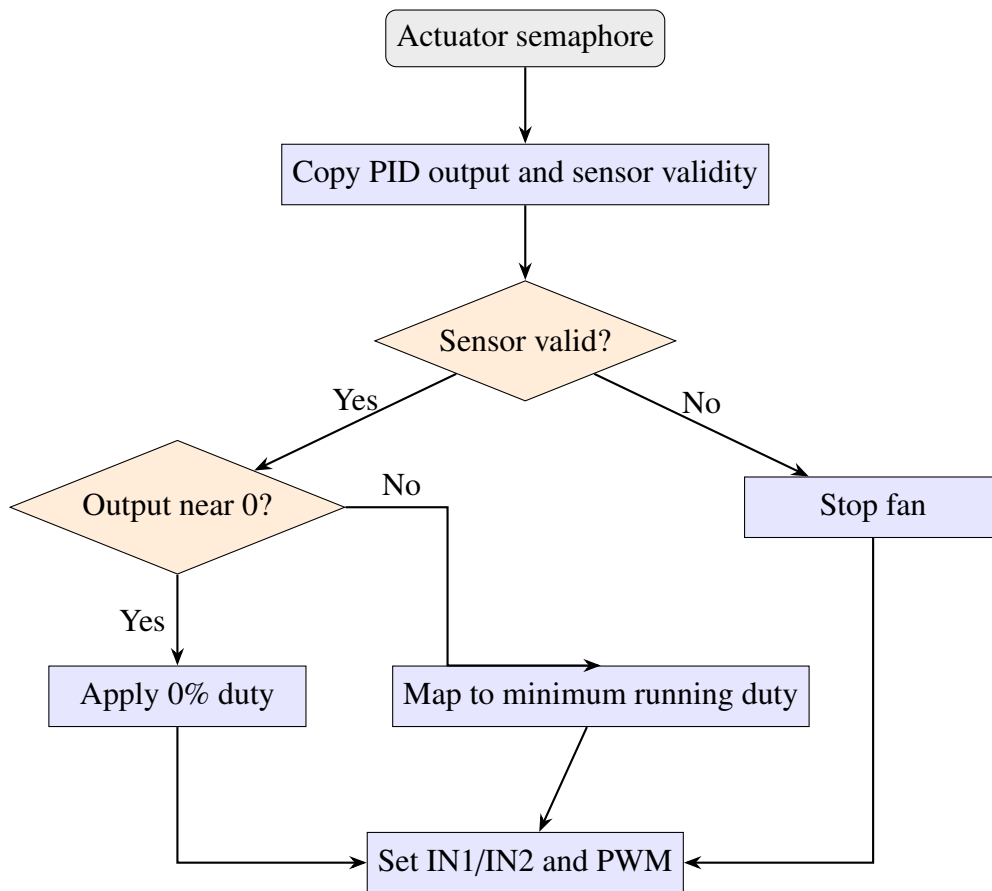


Figure 2.13 – Flowchart of fan duty conditioning and H-bridge actuation

Listing 2.6 – Fan actuation logic

```
float dutyPercent = sensorValid ? mapPidOutputToFanDuty(outputPercent) : 0.0f;
if (dutyPercent > 0.0f) {
    s_fan.setForward(dutyPercent);
} else {
    s_fan.stop();
}
```

Display and Serial Reporting

The display task periodically snapshots the shared state and formats three LCD pages: the main temperature/setpoint page, the PID error/output page, and the gain/duty page. During numeric setpoint entry, the LCD switches to an edit page so the user can see the typed digits before confirmation. The Serial Plotter line uses stable field names: SetPoint, Value, Output, Duty, Error, Kp, Ki, Kd, and Valid. This makes the real hardware run observable without adding a separate debugging protocol.

Timing Configuration

Task	Trigger	Responsibility
Input	50 ms period	Scan debounced keypad events and update setpoint or PID preset.
Acquisition	2000 ms period	Read DHT11 temperature/humidity and potentiometer setpoint.
Control	New-sample semaphore	Apply reverse-action PID and publish bounded output.
Actuation	Actuator semaphore	Convert PID output to PWM fan duty and drive H-bridge pins.
Display	500 ms period	Update LCD pages and print Serial Plotter channels.

Table 2: FreeRTOS task timing and responsibility summary

3. Obtained Results

3.1. Build Process

The PlatformIO environment ‘lab5_2’ compiles successfully for the Arduino Mega 2560 target before the firmware is flashed to the board. The successful build reports approximately 2237 bytes of RAM usage from 8192 bytes and 28662 bytes of flash usage from 253952 bytes, confirming that the PID controller, H-bridge driver, DHT11 wrapper, keypad, LCD, and FreeRTOS task modules are linked correctly.

[Screenshot pending — save successful PlatformIO build output to
resources/ObtainedResults/build_results.png]

Figure 3.1 – Successful PlatformIO build output for ‘lab5_2’

3.2. Physical Hardware Setup

The final validation is intended for the physical circuit shown by the provided wiring image. The setup must show the Arduino Mega, DHT11, potentiometer, keypad, LCD, L293D/L298-style driver, DC fan, shared ground, and external 9 V motor supply. The Wokwi file is useful for checking firmware startup and logic-side pins, but the fan driver and battery branch must be verified on hardware.

[Photo pending — save the complete real Arduino setup to
resources/ObtainedResults/hardware_setup.png]

Figure 3.2 – *Physical Arduino setup prepared for PID fan validation*

3.3. Wokwi Reference Startup

The Wokwi project should be started from ‘labs/wokwi/lab5.2/wokwi.toml’. It is expected to show the LCD, keypad, potentiometer, a DHT-family wiring placeholder, and the D3 PWM proxy. The physical reference run uses a DHT11 on D2; the real H-bridge and 9 V fan branch are represented in the report schematic rather than fully simulated.

[Screenshot pending — save Wokwi startup reference to
resources/ObtainedResults/simulation_start.png]

Figure 3.3 – *Wokwi startup state for the supported Lab 5.2 wiring reference*

3.4. Validation Scenarios

The validation scenarios are derived from the requirements table. Each scenario should be documented with a hardware photo or Serial Plotter screenshot after the real Arduino setup is flashed and running.

Scenario	Action	Expected Evidence
Normal operation	Power the board and wait for DHT11 acquisition.	LCD shows temperature, setpoint, fan duty, and sensor status.
Setpoint change	Rotate the potentiometer or enter a manual keypad setpoint.	‘SetPoint’ changes and the PID error changes sign/magnitude accordingly.
Cooling demand	Warm the DHT11 above the setpoint.	‘Output’ and ‘Duty’ increase, and the fan starts or speeds up.
Near setpoint	Let temperature approach the setpoint.	PID output decreases smoothly rather than switching abruptly.
PID preset change	Press D to cycle SOFT/NORM/FAST.	LCD and Serial Plotter show changed Kp, Ki, and Kd values.
Sensor failure behavior	Disconnect or force an invalid DHT11 read during controlled testing.	‘Valid:0’ appears and fan duty goes to 0%.

Table 3: Hardware validation scenario matrix

Normal Measurement and Display

The LCD should alternate between the primary process page, the error/output page, and the PID gain page. The Serial Plotter line should include ‘SetPoint’, ‘Value’, ‘Output’, ‘Duty’, ‘Error’, ‘Kp’, ‘Ki’, ‘Kd’, and ‘Valid’.

[Screenshot/photo pending — save normal LCD/Serial operation to resources/ObtainedResults/test_normal_operation.png]

Figure 3.4 – Normal PID operation with temperature, setpoint, and fan duty visible

PID Response to Heating

When the DHT11 temperature is raised above the setpoint, the reverse-action PID error becomes positive. The controller should increase its output, and the fan duty should rise after the actuator task converts the PID output to a physical PWM duty.

[Photo/screenshot pending — save heating response validation to resources/ObtainedResults/test_pid_heating_response.png]

Figure 3.5 – PID fan response when measured temperature rises above the setpoint

Manual Setpoint Entry

Digits on the keypad start numeric setpoint entry, '#' confirms the typed value, and '*' cancels the edit. After confirmation, the active setpoint source becomes manual and the Serial Plotter 'SetPoint' channel should reflect the entered value.

[Photo/screenshot pending — save keypad setpoint test to
resources/ObtainedResults/test_manual_setpoint.png]

Figure 3.6 – Manual setpoint entry through the 4x4 keypad

PID Preset Change

Pressing D cycles through the SOFT, NORM, and FAST gain presets. The expected evidence is a visible change in the displayed gains and a different slope or amplitude in the 'Output' channel for the same temperature error.

[Photo/screenshot pending — save PID preset validation to
resources/ObtainedResults/test_pid_preset.png]

Figure 3.7 – PID gain preset selection from the keypad

Serial Plotter PID Trace

The Serial Plotter should show a continuous control trace rather than the binary output of the ON-OFF laboratory. The key evidence is that 'Output' and 'Duty' vary gradually as 'Value' moves relative to 'SetPoint'.

[Screenshot pending — save Arduino Serial Plotter PID trace to
resources/ObtainedResults/serial_plotter_pid.png]

Figure 3.8 – Arduino Serial Plotter trace of PID temperature control

3.5. System Observations

The acquisition period is 2000 ms because the DHT11 should not be polled aggressively. The controller therefore updates at the same rate as the sensor data, while the LCD and Serial Plotter refresh faster so user edits remain visible. The fan duty is mapped from the PID output with a minimum running duty, because small PWM values may not overcome motor friction on real hardware.

The most important limitation is that the DHT11 measures air temperature slowly compared with the electrical response of the fan. Because of that, aggressive PID gains can produce oscillation if the fan cools the sensor area with delay. The preset mechanism supports practical tuning: SOFT is safer for first hardware runs, NORM is the default, and FAST is useful only when the physical setup responds quickly.

3.6. Verification Summary

- **Compilation:** ‘lab5_2’ builds for Arduino Mega 2560.
- **Acquisition:** DHT11 temperature and humidity are sampled in a dedicated task.
- **Setpoint:** A0 potentiometer and keypad manual mode both update the active setpoint.
- **Control:** Reverse-action PID produces a bounded 0–100% output.
- **Actuation:** H-bridge motor driver receives PWM and direction signals from a separate task.
- **Display:** LCD and Serial Plotter expose setpoint, measured value, output, duty, error, gains, and validity.
- **Validation evidence:** Hardware photos and Serial Plotter screenshots must be added after running the real Arduino setup.

4. Conclusion

4.1. Achievement of Objectives

The laboratory work implements a complete PID-based embedded control application for temperature regulation. The system periodically acquires DHT11 temperature data, compares it with a configurable setpoint, computes a reverse-action PID output, and applies the resulting command to a PWM fan actuator through an H-bridge driver. The implementation follows the requested modular structure by separating input, acquisition, control, actuation, display, shared state, and reusable driver modules.

4.2. Performance Analysis and System Limitations

The selected timing model is appropriate for the physical components. The DHT11 is sampled every 2000 ms to respect its practical update rate, while the keypad and LCD are refreshed faster for usability. The control loop is event-driven by the acquisition task, so each PID decision is based on a fresh sample rather than on repeated stale values.

The main limitation is the slow thermal response of the sensor-fan setup. The fan can change speed immediately, but the DHT11 temperature changes gradually and may be affected by airflow placement, room temperature, and sensor self-heating. As a result, PID tuning must be performed carefully on the real circuit. The FAST preset should be used only after the SOFT and NORM presets are verified.

4.3. Proposed Improvements

Future improvements could include storing PID gains in EEPROM, adding a serial command interface for exact gain entry, measuring fan speed with an encoder, and adding an automatic

tuning procedure. A stronger physical setup could also use a real heater element and fan in a controlled chamber, which would produce more repeatable PID response curves.

4.4. Reflections on the Learning Experience

This laboratory highlights the difference between binary ON-OFF control and continuous PID control. The firmware must track previous error, accumulated error, sampling time, actuator limits, and safe behavior on invalid sensor reads. The project also shows why embedded control code benefits from strict modularity: tuning the PID controller, changing the fan driver, or replacing the input method can be done without rewriting the whole application.

4.5. Impact of Technology in Real-World Applications

PID control is one of the most common control strategies in industrial automation, HVAC, robotics, and process control. Even this small Arduino system reflects real engineering concerns: sensor update limits, actuator saturation, power-stage isolation, diagnostic display, and real-time data plotting. The same architecture can be scaled to regulate greenhouse ventilation, electronics cooling, incubator temperature, or motor speed in embedded products.

5. Note Regarding Usage of AI

During the writing of this report, the author used AI assistance to generate and consolidate explanatory text, LaTeX diagrams, and implementation descriptions. The generated content was reviewed, validated, and adjusted according to the laboratory requirements, the physical wiring constraints, and the final PlatformIO implementation.

6. Bibliography

- [1] Bragarenco, A., Astafi, V., *Sisteme Electronice Încorporate: Indicații metodice pentru lucrări de laborator*, Universitatea Tehnică a Moldovei, Chișinău, 2024.
- [2] Arduino, *Arduino Mega 2560 Rev3 Documentation*, <https://docs.arduino.cc/hardware/mega-2560/>, Accessed: 2026-04-27.
- [3] FreeRTOS, *FreeRTOS Kernel Documentation*, https://www.freertos.org/Documentation/RTOS_book.html, Accessed: 2026-04-27.
- [4] Adafruit, *DHT Sensor Library*, <https://github.com/adafruit/DHT-sensor-library>, Accessed: 2026-04-27.
- [5] Wokwi, *Wokwi Arduino Mega Simulator Documentation*, <https://docs.wokwi.com/parts/wokwi-arduino-mega>, Accessed: 2026-04-27.

- [6] Texas Instruments, *L293x Quadruple Half-H Drivers Datasheet*, <https://www.ti.com/lit/ds/symlink/l293.pdf>, Accessed: 2026-04-27.

7. Appendix

The complete source code is organized below by architectural layer. All source files are available in the project GitHub repository.

GitHub Repository: <https://github.com/mcittkmims/es-labs>

7.1. Application Layer

Listing 7.1 – Lab 5.2 entry point header

```
/**
 * @file lab5_2_main.h
 * @brief Lab 5.2 - PID temperature control entry point.
 */

#ifndef LAB5_2_MAIN_H
#define LAB5_2_MAIN_H

void lab5_2Setup();
void lab5_2Loop();

#endif // LAB5_2_MAIN_H
```

Listing 7.2 – Lab 5.2 entry point implementation

```
/**
 * @file lab5_2_main.cpp
 * @brief Lab 5.2 - PID temperature control with PWM fan actuator.
 */

#include "lab5_2_main.h"
#include "lab5_2_config.h"
#include "shared_state.h"
#include "task_input.h"
#include "task_acquisition.h"
#include "task_control.h"
#include "task_actuation.h"
#include "task_display.h"

#include <Arduino.h>
#include <Arduino_FreeRTOS.h>
#include <stdio.h>

#include "StdioSerial.h"
```

```

void lab5_2Setup() {
    stdioSerialInit(9600);

    printf("\r\n");
    printf("=====\r\n");
    printf("  Lab 5.2 - PID Control\r\n");
    printf("  Variant: DHT11 temperature + PWM fan via L293D\r\n");
    printf("  Arduino Mega | FreeRTOS | LCD | Keypad\r\n");
    printf("=====\r\n");
    printf("KEYPAD:\r\n");
    printf("  A = toggle setpoint source POT/MANUAL\r\n");
    printf("  B/C = decrease/increase manual setpoint by 0.5 C\r\n");
    printf("  D = cycle PID preset SOFT/NORM/FAST\r\n");
    printf("  digits + # = enter integer manual setpoint\r\n");
    printf("  * = cancel numeric entry\r\n");
    printf("PINS:\r\n");
    printf("  DHT11 data: D%u\r\n", (unsigned)PIN_DHT_SENSOR);
    printf("  Fan EN/PWM: D%u\r\n", (unsigned)PIN_FAN_PWM);
    printf("  Fan IN1:    D%u\r\n", (unsigned)PIN_FAN_IN1);
    printf("  Fan IN2:    D%u\r\n", (unsigned)PIN_FAN_IN2);
    printf("  Pot SIG:    A0\r\n");
    printf("  LCD:        SDA/SCL\r\n");
    printf("PLOTTER LINE:\r\n");
    printf("  SetPoint:<C> Value:<C> Output:<%%> Duty:<%%> Error:<C> Kp Ki Kd Valid\r\n");
    ;
    printf("=====\r\n\r\n");

    lab5PidStateInit();

    BaseType_t okInput = xTaskCreate(
        vTaskLab5PidInput,
        "Input",
        TASK_INPUT_STACK,
        NULL,
        TASK_INPUT_PRIORITY,
        NULL
    );

    BaseType_t okAcquisition = xTaskCreate(
        vTaskLab5PidAcquisition,
        "Acquire",
        TASK_ACQUISITION_STACK,
        NULL,
        TASK_ACQUISITION_PRIORITY,
        NULL
    );
}

```

```

BaseType_t okControl = xTaskCreate(
    vTaskLab5PidControl,
    "Control",
    TASK_CONTROL_STACK,
    NULL,
    TASK_CONTROL_PRIORITY,
    NULL
);

BaseType_t okActuation = xTaskCreate(
    vTaskLab5PidActuation,
    "Actuate",
    TASK_ACTUATION_STACK,
    NULL,
    TASK_ACTUATION_PRIORITY,
    NULL
);

BaseType_t okDisplay = xTaskCreate(
    vTaskLab5PidDisplay,
    "Display",
    TASK_DISPLAY_STACK,
    NULL,
    TASK_DISPLAY_PRIORITY,
    NULL
);

if (okInput != pdPASS || okAcquisition != pdPASS ||
    okControl != pdPASS || okActuation != pdPASS ||
    okDisplay != pdPASS) {
    printf("[ERROR] Task creation failed: I=%ld A=%ld C=%ld M=%ld D=%ld\r\n",
        (long)okInput,
        (long)okAcquisition,
        (long)okControl,
        (long)okActuation,
        (long)okDisplay);
}
}

void lab5_2Loop() {
    // All application logic runs in FreeRTOS tasks.
}

```

Listing 7.3 – Lab 5.2 configuration header

```

/**
 * @file lab5_2_config.h
 * @brief Lab 5.2 configuration and pin mapping.
 */

```

```

#ifndef LAB5_2_CONFIG_H
#define LAB5_2_CONFIG_H

#include <Arduino.h>
#include <Arduino_FreeRTOS.h>
#include <DHT.h>

static const uint8_t PIN_DHT_SENSOR = 2;
static const uint8_t DHT_SENSOR_TYPE = DHT11;
static const uint8_t PIN_SETPOINT_POT = A0;

// Reference physical fan branch: L293D/L298-style H-bridge.
static const uint8_t PIN_FAN_PWM = 3;
static const uint8_t PIN_FAN_IN1 = 16;
static const uint8_t PIN_FAN_IN2 = 17;

extern byte KEYPAD_ROW_PINS[4];
extern byte KEYPAD_COL_PINS[4];

static const uint8_t LCD_I2C_ADDRESS = 0x27;
static const uint8_t LCD_COLS = 16;
static const uint8_t LCD_ROWS = 2;

static const float SETPOINT_MIN_C = 18.0f;
static const float SETPOINT_MAX_C = 35.0f;
static const float SETPOINT_DEFAULT_C = 25.0f;
static const float SETPOINT_STEP_C = 0.5f;
static const uint8_t SETPOINT_INPUT_MAX_DIGITS = 3;

static const float PID_OUTPUT_MIN_PERCENT = 0.0f;
static const float PID_OUTPUT_MAX_PERCENT = 100.0f;
static const float FAN_MIN_RUNNING_DUTY_PERCENT = 30.0f;
static const float FAN_STOP_THRESHOLD_PERCENT = 1.0f;

static const float PID_DEFAULT_KP = 12.0f;
static const float PID_DEFAULT_KI = 0.18f;
static const float PID_DEFAULT_KD = 2.0f;

struct PidPreset {
    const char *name;
    float kp;
    float ki;
    float kd;
};

static const uint8_t PID_PRESET_COUNT = 3;
extern const PidPreset PID_PRESETS[PID_PRESET_COUNT];

```

```

static const uint16_t TASK_INPUT_PERIOD_MS = 50;
static const uint16_t TASK_ACQUISITION_PERIOD_MS = 2000;
static const uint16_t TASK_DISPLAY_PERIOD_MS = 500;

static const configSTACK_DEPTH_TYPE TASK_INPUT_STACK = 256;
static const configSTACK_DEPTH_TYPE TASK_ACQUISITION_STACK = 384;
static const configSTACK_DEPTH_TYPE TASK_CONTROL_STACK = 384;
static const configSTACK_DEPTH_TYPE TASK_ACTUATION_STACK = 256;
static const configSTACK_DEPTH_TYPE TASK_DISPLAY_STACK = 704;

static const UBaseType_t TASK_INPUT_PRIORITY = 3;
static const UBaseType_t TASK_ACQUISITION_PRIORITY = 3;
static const UBaseType_t TASK_CONTROL_PRIORITY = 2;
static const UBaseType_t TASK_ACTUATION_PRIORITY = 2;
static const UBaseType_t TASK_DISPLAY_PRIORITY = 1;

#endif // LAB5_2_CONFIG_H

```

Listing 7.4 – Lab 5.2 configuration implementation

```

/**
 * @file lab5_2_config.cpp
 * @brief Lab 5.2 writable configuration arrays and PID presets.
 */

#include "lab5_2_config.h"

byte KEYPAD_ROW_PINS[4] = {11, 10, 9, 8};
byte KEYPAD_COL_PINS[4] = {7, 6, 5, 4};

const PidPreset PID_PRESETS[PID_PRESET_COUNT] = {
    {"SOFT", 8.0f, 0.10f, 1.0f},
    {"NORM", PID_DEFAULT_KP, PID_DEFAULT_KI, PID_DEFAULT_KD},
    {"FAST", 18.0f, 0.30f, 3.0f}
};

```

7.2. Service Layer

Listing 7.5 – Shared PID state header

```

/**
 * @file shared_state.h
 * @brief Lab 5.2 shared PID control state.
 */

#ifndef LAB5_2_SHARED_STATE_H
#define LAB5_2_SHARED_STATE_H

```

```

#include <Arduino.h>
#include <Arduino_FreeRTOS.h>
#include <semphr.h>
#include "lab5_2_config.h"

enum SetpointSource {
    SETPOINT_SOURCE_POT = 0,
    SETPOINT_SOURCE_MANUAL = 1
};

struct Lab5PidState {
    float measuredTempC;
    float measuredHumidityPercent;
    bool sensorValid;
    uint16_t potRaw;
    float potSetpointC;
    float manualSetpointC;
    float activeSetpointC;
    SetpointSource setpointSource;

    float kp;
    float ki;
    float kd;
    uint8_t pidPresetIndex;

    float errorC;
    float pidIntegral;
    float pidDerivative;
    float controlOutputPercent;
    float appliedDutyPercent;
    bool fanRunning;

    bool editingSetpoint;
    char inputBuffer[SETPOINT_INPUT_MAX_DIGITS + 1];
    uint8_t inputBufferLen;

    uint32_t sampleCount;
    uint32_t controlCycles;
    uint32_t actuatorUpdates;
    TickType_t lastSampleTick;
};

extern Lab5PidState g_lab5PidState;
extern SemaphoreHandle_t xLab5PidStateMutex;
extern SemaphoreHandle_t xLab5PidNewSampleSemaphore;
extern SemaphoreHandle_t xLab5PidActuatorSemaphore;

```

```

void lab5PidStateInit();
void lab5PidStateLock();
void lab5PidStateUnlock();
Lab5PidState* lab5PidStateGet();

#endif // LAB5_2_SHARED_STATE_H

```

Listing 7.6 – Shared PID state implementation

```

/**
 * @file shared_state.cpp
 * @brief Lab 5.2 shared state implementation.
 */

#include "shared_state.h"
#include <math.h>
#include <string.h>

Lab5PidState g_lab5PidState;
SemaphoreHandle_t xLab5PidStateMutex = NULL;
SemaphoreHandle_t xLab5PidNewSampleSemaphore = NULL;
SemaphoreHandle_t xLab5PidActuatorSemaphore = NULL;

void lab5PidStateInit() {
    memset(&g_lab5PidState, 0, sizeof(g_lab5PidState));

    g_lab5PidState.measuredTempC = NAN;
    g_lab5PidState.measuredHumidityPercent = NAN;
    g_lab5PidState.sensorValid = false;
    g_lab5PidState.potRaw = 0;
    g_lab5PidState.potSetpointC = SETPOINT_DEFAULT_C;
    g_lab5PidState.manualSetpointC = SETPOINT_DEFAULT_C;
    g_lab5PidState.activeSetpointC = SETPOINT_DEFAULT_C;
    g_lab5PidState.setpointSource = SETPOINT_SOURCE_POT;

    g_lab5PidState.pidPresetIndex = 1;
    g_lab5PidState.kp = PID_PRESETS[g_lab5PidState.pidPresetIndex].kp;
    g_lab5PidState.ki = PID_PRESETS[g_lab5PidState.pidPresetIndex].ki;
    g_lab5PidState.kd = PID_PRESETS[g_lab5PidState.pidPresetIndex].kd;

    g_lab5PidState.errorC = 0.0f;
    g_lab5PidState.pidIntegral = 0.0f;
    g_lab5PidState.pidDerivative = 0.0f;
    g_lab5PidState.controlOutputPercent = 0.0f;
    g_lab5PidState.appliedDutyPercent = 0.0f;
    g_lab5PidState.fanRunning = false;
    g_lab5PidState.editingSetpoint = false;
    g_lab5PidState.inputBuffer[0] = '\0';
    g_lab5PidState.inputBufferLen = 0;

```

```

    xLab5PidStateMutex = xSemaphoreCreateMutex();
    xLab5PidNewSampleSemaphore = xSemaphoreCreateBinary();
    xLab5PidActuatorSemaphore = xSemaphoreCreateBinary();
}

void lab5PidStateLock() {
    xSemaphoreTake(xLab5PidStateMutex, portMAX_DELAY);
}

void lab5PidStateUnlock() {
    xSemaphoreGive(xLab5PidStateMutex);
}

Lab5PidState* lab5PidStateGet() {
    return &g_lab5PidState;
}

```

Listing 7.7 – Keypad input task

```

/**
 * @file task_input.cpp
 * @brief Lab 5.2 keypad input task implementation.
 */

#include "task_input.h"
#include "lab5_2_config.h"
#include "shared_state.h"
#include "KeypadInput.h"

#include <Arduino_FreeRTOS.h>
#include <stdlib.h>
#include <stdio.h>

static KeypadInput s_keypad(KEYPAD_ROW_PINS, KEYPAD_COL_PINS);

static float clampFloat(float value, float minValue, float maxValue) {
    if (value < minValue) return minValue;
    if (value > maxValue) return maxValue;
    return value;
}

static void enterManualMode(Lab5PidState *state) {
    if (state->setpointSource == SETPOINT_SOURCE_POT) {
        state->>manualSetpointC = state->activeSetpointC;
    }
    state->setpointSource = SETPOINT_SOURCE_MANUAL;
    state->activeSetpointC = state->>manualSetpointC;
}

```



```

static void applyPidPreset(Lab5PidState *state, uint8_t presetIndex) {
    if (presetIndex >= PID_PRESET_COUNT) {
        presetIndex = 0;
    }

    state->pidPresetIndex = presetIndex;
    state->kp = PID_PRESETS[presetIndex].kp;
    state->ki = PID_PRESETS[presetIndex].ki;
    state->kd = PID_PRESETS[presetIndex].kd;
}

void vTaskLab5PidInput(void *pvParameters) {
    (void)pvParameters;

    s_keypad.init();

    TickType_t lastWake = xTaskGetTickCount();
    const TickType_t period = pdMS_TO_TICKS(TASK_INPUT_PERIOD_MS);

    for (;;) {
        char key = s_keypad.getKey();

        if (key != 0) {
            lab5PidStateLock();
            Lab5PidState *state = lab5PidStateGet();

            switch (key) {
                case 'A':
                    if (state->setpointSource == SETPOINT_SOURCE_POT) {
                        enterManualMode(state);
                        printf("[INPUT] Setpoint source: MANUAL\r\n");
                    } else {
                        state->setpointSource = SETPOINT_SOURCE_POT;
                        state->activeSetpointC = state->potSetpointC;
                        printf("[INPUT] Setpoint source: POT\r\n");
                    }
                    state->editingSetpoint = false;
                    state->inputBufferLen = 0;
                    state->inputBuffer[0] = '\0';
                    break;

                case 'B':
                    enterManualMode(state);
                    state->manualSetpointC = clampFloat(
                        state->manualSetpointC - SETPOINT_STEP_C,
                        SETPOINT_MIN_C,
                        SETPOINT_MAX_C
                    );

```

```

    );
    state->activeSetpointC = state->manualSetpointC;
    printf("[INPUT] Manual setpoint decreased\r\n");
    break;

case 'C':
    enterManualMode(state);
    state->manualSetpointC = clampFloat(
        state->manualSetpointC + SETPOINT_STEP_C,
        SETPOINT_MIN_C,
        SETPOINT_MAX_C
    );
    state->activeSetpointC = state->manualSetpointC;
    printf("[INPUT] Manual setpoint increased\r\n");
    break;

case 'D': {
    uint8_t nextPreset = state->pidPresetIndex + 1;
    if (nextPreset >= PID_PRESET_COUNT) {
        nextPreset = 0;
    }
    applyPidPreset(state, nextPreset);
    printf("[INPUT] PID preset: %s\r\n",
        PID_PRESETS[nextPreset].name);
    break;
}

case '*':
    state->editingSetpoint = false;
    state->inputBufferLen = 0;
    state->inputBuffer[0] = '\0';
    printf("[INPUT] Numeric setpoint entry cancelled\r\n");
    break;

case '#':
    if (state->editingSetpoint && state->inputBufferLen > 0) {
        state->inputBuffer[state->inputBufferLen] = '\0';
        int requested = atoi(state->inputBuffer);
        state->manualSetpointC = clampFloat(
            (float)requested,
            SETPOINT_MIN_C,
            SETPOINT_MAX_C
        );
        state->setpointSource = SETPOINT_SOURCE_MANUAL;
        state->activeSetpointC = state->manualSetpointC;
        state->editingSetpoint = false;
        state->inputBufferLen = 0;
        state->inputBuffer[0] = '\0';
    }
}

```

```

        printf("[INPUT] Manual setpoint confirmed\r\n");
    }
    break;

default:
    if (key >= '0' && key <= '9') {
        if (!state->editingSetpoint) {
            state->editingSetpoint = true;
            state->inputBufferLen = 0;
            state->inputBuffer[0] = '\0';
        }

        if (state->inputBufferLen < SETPOINT_INPUT_MAX_DIGITS) {
            state->inputBuffer[state->inputBufferLen++] = key;
            state->inputBuffer[state->inputBufferLen] = '\0';
            printf("[INPUT] Entering setpoint: %s\r\n",
                state->inputBuffer);
        }
    }
    break;
}

lab5PidStateUnlock();
}

vTaskDelayUntil(&lastWake, period);
}
}

```

Listing 7.8 – Acquisition task

```

/**
 * @file task_acquisition.cpp
 * @brief Lab 5.2 DHT11 and potentiometer acquisition task.
 */

#include "task_acquisition.h"
#include "lab5_2_config.h"
#include "shared_state.h"

#include "DhtSensor.h"
#include "AnalogSetpointInput.h"

#include <Arduino_FreeRTOS.h>
#include <DHT.h>

static DhtSensor s_dht(PIN_DHT_SENSOR, DHT_SENSOR_TYPE);
static AnalogSetpointInput s_setpointPot(
    PIN_SETPOINT_POT,

```

```

    SETPOINT_MIN_C,
    SETPOINT_MAX_C
);

void vTaskLab5PidAcquisition(void *pvParameters) {
    (void)pvParameters;

    s_dht.init();
    s_setpointPot.init();

    TickType_t lastWake = xTaskGetTickCount();
    const TickType_t period = pdMS_TO_TICKS(TASK_ACQUISITION_PERIOD_MS);

    for (;;) {
        bool sensorOk = s_dht.read();
        float temperature = s_dht.getTemperatureC();
        float humidity = s_dht.getHumidityPercent();
        float potSetpoint = s_setpointPot.readValue();
        uint16_t potRaw = s_setpointPot.getLastRaw();

        lab5PidStateLock();
        Lab5PidState *state = lab5PidStateGet();

        state->sensorValid = sensorOk;
        state->measuredTempC = temperature;
        state->measuredHumidityPercent = humidity;
        state->potRaw = potRaw;
        state->potSetpointC = potSetpoint;

        if (state->setpointSource == SETPOINT_SOURCE_POT) {
            state->activeSetpointC = potSetpoint;
        } else {
            state->activeSetpointC = state->>manualSetpointC;
        }

        state->sampleCount++;
        state->lastSampleTick = xTaskGetTickCount();

        lab5PidStateUnlock();
        xSemaphoreGive(xLab5PidNewSampleSemaphore);

        vTaskDelayUntil(&lastWake, period);
    }
}

```

Listing 7.9 – PID control task

```

/**
 * @file task_control.cpp

```

```

* @brief Lab 5.2 PID control task implementation.
*/

#include "task_control.h"
#include "lab5_2_config.h"
#include "shared_state.h"
#include "PidController.h"

#include <Arduino_FreeRTOS.h>
#include <math.h>

static PidController s_pid(
    PID_DEFAULT_KP,
    PID_DEFAULT_KI,
    PID_DEFAULT_KD,
    PID_OUTPUT_MIN_PERCENT,
    PID_OUTPUT_MAX_PERCENT,
    PID_REVERSE
);

static float elapsedSeconds(TickType_t previousTick, TickType_t currentTick) {
    if (previousTick == 0 || currentTick <= previousTick) {
        return (float)TASK_ACQUISITION_PERIOD_MS / 1000.0f;
    }

    TickType_t deltaTicks = currentTick - previousTick;
    return ((float)deltaTicks * (float)portTICK_PERIOD_MS) / 1000.0f;
}

void vTaskLab5PidControl(void *pvParameters) {
    (void)pvParameters;

    s_pid.init();
    TickType_t previousControlTick = 0;

    for (;;) {
        if (xSemaphoreTake(xLab5PidNewSampleSemaphore, portMAX_DELAY) != pdTRUE) {
            continue;
        }

        TickType_t now = xTaskGetTickCount();
        float dtSeconds = elapsedSeconds(previousControlTick, now);
        previousControlTick = now;

        lab5PidStateLock();
        Lab5PidState *state = lab5PidStateGet();
        float temperature = state->measuredTempC;
        float setpoint = state->activeSetpointC;
    }
}

```

```

    float kp = state->kp;
    float ki = state->ki;
    float kd = state->kd;
    bool valid = state->sensorValid && !isnan(temperature);
    lab5PidStateUnlock();

    s_pid.setTunings(kp, ki, kd);

    float output = 0.0f;
    if (valid) {
        output = s_pid.update(setpoint, temperature, dtSeconds);
    } else {
        s_pid.reset();
    }

    lab5PidStateLock();
    state = lab5PidStateGet();
    state->controlOutputPercent = output;
    state->errorC = valid ? s_pid.getError() : 0.0f;
    state->pidIntegral = s_pid.getIntegral();
    state->pidDerivative = s_pid.getDerivative();
    state->controlCycles++;
    lab5PidStateUnlock();

    xSemaphoreGive(xLab5PidActuatorSemaphore);
}
}

```

Listing 7.10 – Fan actuation task

```

/**
 * @file task_actuation.cpp
 * @brief Lab 5.2 fan actuation task implementation.
 */

#include "task_actuation.h"
#include "lab5_2_config.h"
#include "shared_state.h"
#include "HBridgeMotor.h"

#include <Arduino_FreeRTOS.h>

static HBridgeMotor s_fan(PIN_FAN_PWM, PIN_FAN_IN1, PIN_FAN_IN2);

static float mapPidOutputToFanDuty(float outputPercent) {
    if (outputPercent <= FAN_STOP_THRESHOLD_PERCENT) {
        return 0.0f;
    }
}

```

```

float usableRange = 100.0f - FAN_MIN_RUNNING_DUTY_PERCENT;
return FAN_MIN_RUNNING_DUTY_PERCENT +
       (outputPercent * usableRange / 100.0f);
}

void vTaskLab5PidActuation(void *pvParameters) {
    (void)pvParameters;

    s_fan.init();

    lab5PidStateLock();
    lab5PidStateGet()->appliedDutyPercent = s_fan.getDuty();
    lab5PidStateGet()->fanRunning = false;
    lab5PidStateUnlock();

    for (;;) {
        if (xSemaphoreTake(xLab5PidActuatorSemaphore, portMAX_DELAY) != pdTRUE) {
            continue;
        }

        lab5PidStateLock();
        Lab5PidState *state = lab5PidStateGet();
        float outputPercent = state->controlOutputPercent;
        bool sensorValid = state->sensorValid;
        lab5PidStateUnlock();

        float dutyPercent = sensorValid ? mapPidOutputToFanDuty(outputPercent) : 0.0f;
        if (dutyPercent > 0.0f) {
            s_fan.setForward(dutyPercent);
        } else {
            s_fan.stop();
        }

        lab5PidStateLock();
        state = lab5PidStateGet();
        state->appliedDutyPercent = s_fan.getDuty();
        state->fanRunning = s_fan.getDuty() > 0.0f;
        state->actuatorUpdates++;
        lab5PidStateUnlock();
    }
}

```

Listing 7.11 – LCD and Serial Plotter task

```

/**
 * @file task_display.cpp
 * @brief Lab 5.2 LCD and Serial Plotter reporting task.
 */

```

```

#include "task_display.h"
#include "lab5_2_config.h"
#include "shared_state.h"
#include "LcdDisplay.h"

#include <Arduino_FreeRTOS.h>
#include <math.h>
#include <stdio.h>
#include <string.h>

static LcdDisplay s_lcd(LCD_I2C_ADDRESS, LCD_COLS, LCD_ROWS);

static void formatFloat(char *buffer, size_t size, float value,
                        signed char width, unsigned char precision,
                        const char *invalidText) {
    if (isnan(value)) {
        strncpy(buffer, invalidText, size);
        buffer[size - 1] = '\0';
    } else {
        dtostrf(value, width, precision, buffer);
    }
}

static float plotValueOrZero(float value) {
    return isnan(value) ? 0.0f : value;
}

void vTaskLab5PidDisplay(void *pvParameters) {
    (void)pvParameters;

    s_lcd.init();
    s_lcd.backlight(true);
    s_lcd.showTwoLines("Lab 5.2 PID", "DHT11 + Fan");

    TickType_t lastWake = xTaskGetTickCount();
    const TickType_t period = pdMS_TO_TICKS(TASK_DISPLAY_PERIOD_MS);
    uint8_t displayCycle = 0;

    for (;;) {
        vTaskDelayUntil(&lastWake, period);
        displayCycle++;

        lab5PidStateLock();
        Lab5PidState snapshot = *lab5PidStateGet();
        lab5PidStateUnlock();

        char tempStr[8];
        char spStr[8];
    }
}

```



```

char outStr[8];
char errStr[8];
char kpStr[8];
char kiStr[8];
char kdStr[8];

formatFloat(tempStr, sizeof(tempStr), snapshot.measuredTempC,
            4, 1, "--.-");
formatFloat(spStr, sizeof(spStr), snapshot.activeSetpointC,
            4, 1, "--.-");
formatFloat(outStr, sizeof(outStr), snapshot.appliedDutyPercent,
            4, 0, "---");
formatFloat(errStr, sizeof(errStr), snapshot.errorC,
            4, 1, "-.-");
formatFloat(kpStr, sizeof(kpStr), snapshot.kp,
            4, 1, "-.-");
formatFloat(kiStr, sizeof(kiStr), snapshot.ki,
            4, 2, "-.-");
formatFloat(kdStr, sizeof(kdStr), snapshot.kd,
            4, 1, "-.-");

char line0[17];
char line1[17];

if (snapshot.editingSetpoint) {
    snprintf(line0, sizeof(line0), "Set SP:%-3s C", snapshot.inputBuffer);
    snprintf(line1, sizeof(line1), "#=OK *=CLR");
} else if (((displayCycle / 4) % 3) == 0) {
    snprintf(line0, sizeof(line0), "T:%s SP:%s", tempStr, spStr);
    snprintf(line1, sizeof(line1), "Fan:%s% %s",
            outStr,
            snapshot.sensorValid ? "OK" : "SERR");
} else if (((displayCycle / 4) % 3) == 1) {
    const char *source =
        snapshot.setpointSource == SETPOINT_SOURCE_POT ? "POT" : "MAN";
    snprintf(line0, sizeof(line0), "Err:%s Out:%s", errStr, outStr);
    snprintf(line1, sizeof(line1), "%s %s",
            source,
            PID_PRESETS[snapshot.pidPresetIndex].name);
} else {
    snprintf(line0, sizeof(line0), "P:%s I:%s", kpStr, kiStr);
    snprintf(line1, sizeof(line1), "D:%s Duty:%s", kdStr, outStr);
}

s_lcd.showTwoLines(line0, line1);

char plotSetpoint[10];
char plotValue[10];

```

```

    char plotOutput[10];
    char plotDuty[10];
    char plotError[10];
    char plotKp[10];
    char plotKi[10];
    char plotKd[10];

    dtostrf(plotValueOrZero(snapshot.activeSetpointC), 1, 1, plotSetpoint);
    dtostrf(plotValueOrZero(snapshot.measuredTempC), 1, 1, plotValue);
    dtostrf(plotValueOrZero(snapshot.controlOutputPercent), 1, 1, plotOutput);
    dtostrf(plotValueOrZero(snapshot.appliedDutyPercent), 1, 1, plotDuty);
    dtostrf(plotValueOrZero(snapshot.errorC), 1, 1, plotError);
    dtostrf(plotValueOrZero(snapshot.kp), 1, 1, plotKp);
    dtostrf(plotValueOrZero(snapshot.ki), 1, 2, plotKi);
    dtostrf(plotValueOrZero(snapshot.kd), 1, 1, plotKd);

    printf("SetPoint:%s Value:%s Output:%s Duty:%s Error:%s Kp:%s Ki:%s Kd:%s Valid:%s\n",
           plotSetpoint,
           plotValue,
           plotOutput,
           plotDuty,
           plotError,
           plotKp,
           plotKi,
           plotKd,
           snapshot.sensorValid ? 1U : 0U);
}
}

```

7.3. Reusable Control and Driver Modules

Listing 7.12 – PID controller interface

```

/**
 * @file PidController.h
 * @brief Reusable discrete PID controller.
 */

#ifndef PID_CONTROLLER_H
#define PID_CONTROLLER_H

enum PidDirection {
    PID_DIRECT = 0,
    PID_REVERSE = 1
};

class PidController {

```

```

public:
    PidController(float kp, float ki, float kd,
                  float outputMin, float outputMax,
                  PidDirection direction = PID_DIRECT);

    void init();
    void setTunings(float kp, float ki, float kd);
    void setOutputLimits(float outputMin, float outputMax);
    void setDirection(PidDirection direction);
    void reset();

    float update(float setpoint, float measuredValue, float dtSeconds);

    float getKp() const;
    float getKi() const;
    float getKd() const;
    float getError() const;
    float getIntegral() const;
    float getDerivative() const;
    float getOutput() const;

private:
    float calculateError(float setpoint, float measuredValue) const;
    float clamp(float value, float minValue, float maxValue) const;

    float _kp;
    float _ki;
    float _kd;
    float _outputMin;
    float _outputMax;
    PidDirection _direction;

    bool _hasPreviousError;
    float _previousError;
    float _integral;
    float _derivative;
    float _lastError;
    float _lastOutput;
};

#endif // PID_CONTROLLER_H

```

Listing 7.13 – PID controller implementation

```

/**
 * @file PidController.cpp
 * @brief Reusable discrete PID controller implementation.
 */

```

```

#include "PidController.h"

PidController::PidController(float kp, float ki, float kd,
                             float outputMin, float outputMax,
                             PidDirection direction)
    : _kp(kp),
      _ki(ki),
      _kd(kd),
      _outputMin(outputMin),
      _outputMax(outputMax),
      _direction(direction),
      _hasPreviousError(false),
      _previousError(0.0f),
      _integral(0.0f),
      _derivative(0.0f),
      _lastError(0.0f),
      _lastOutput(0.0f) {}

void PidController::init() {
    reset();
}

void PidController::setTunings(float kp, float ki, float kd) {
    if (kp < 0.0f) kp = 0.0f;
    if (ki < 0.0f) ki = 0.0f;
    if (kd < 0.0f) kd = 0.0f;
    _kp = kp;
    _ki = ki;
    _kd = kd;
}

void PidController::setOutputLimits(float outputMin, float outputMax) {
    if (outputMax < outputMin) {
        float tmp = outputMax;
        outputMax = outputMin;
        outputMin = tmp;
    }

    _outputMin = outputMin;
    _outputMax = outputMax;
    _integral = clamp(_integral, _outputMin, _outputMax);
    _lastOutput = clamp(_lastOutput, _outputMin, _outputMax);
}

void PidController::setDirection(PidDirection direction) {
    _direction = direction;
}

```

```

void PidController::reset() {
    _hasPreviousError = false;
    _previousError = 0.0f;
    _integral = 0.0f;
    _derivative = 0.0f;
    _lastError = 0.0f;
    _lastOutput = 0.0f;
}

float PidController::update(float setpoint, float measuredValue, float dtSeconds) {
    if (dtSeconds <= 0.0f) {
        dtSeconds = 0.001f;
    }

    float error = calculateError(setpoint, measuredValue);
    float proportional = _kp * error;

    _integral += _ki * error * dtSeconds;
    _integral = clamp(_integral, _outputMin, _outputMax);

    if (_hasPreviousError) {
        _derivative = (error - _previousError) / dtSeconds;
    } else {
        _derivative = 0.0f;
        _hasPreviousError = true;
    }

    float derivativeTerm = _kd * _derivative;
    float output = proportional + _integral + derivativeTerm;
    output = clamp(output, _outputMin, _outputMax);

    _previousError = error;
    _lastError = error;
    _lastOutput = output;
    return output;
}

float PidController::getKp() const {
    return _kp;
}

float PidController::getKi() const {
    return _ki;
}

float PidController::getKd() const {
    return _kd;
}

```

```

float PidController::getError() const {
    return _lastError;
}

float PidController::getIntegral() const {
    return _integral;
}

float PidController::getDerivative() const {
    return _derivative;
}

float PidController::getOutput() const {
    return _lastOutput;
}

float PidController::calculateError(float setpoint, float measuredValue) const {
    if (_direction == PID_REVERSE) {
        return measuredValue - setpoint;
    }
    return setpoint - measuredValue;
}

float PidController::clamp(float value, float minValue, float maxValue) const {
    if (value < minValue) return minValue;
    if (value > maxValue) return maxValue;
    return value;
}

```

Listing 7.14 – H-bridge motor interface

```

/**
 * @file HBridgeMotor.h
 * @brief Unidirectional/bidirectional DC motor driver for an H-bridge.
 */

#ifndef HBRIDGE_MOTOR_H
#define HBRIDGE_MOTOR_H

#include <Arduino.h>
#include "PwmActuator.h"

enum HBridgeDirection {
    HBRIDGE_STOPPED = 0,
    HBRIDGE_FORWARD = 1,
    HBRIDGE_REVERSE = 2
};

```

```

class HBridgeMotor {
public:
    HBridgeMotor(uint8_t enablePwmPin, uint8_t input1Pin, uint8_t input2Pin);

    void init();
    void setForward(float dutyPercent);
    void setReverse(float dutyPercent);
    void stop();

    float getDuty() const;
    uint8_t getRawPwm() const;
    HBridgeDirection getDirection() const;

private:
    void setDirectionPins(HBridgeDirection direction);

    PwmActuator _enable;
    uint8_t _input1Pin;
    uint8_t _input2Pin;
    HBridgeDirection _direction;
};

#endif // HBRIDGE_MOTOR_H

```

Listing 7.15 – H-bridge motor implementation

```

/**
 * @file HBridgeMotor.cpp
 * @brief H-bridge DC motor driver implementation.
 */

#include "HBridgeMotor.h"

HBridgeMotor::HBridgeMotor(uint8_t enablePwmPin, uint8_t input1Pin, uint8_t input2Pin)
    : _enable(enablePwmPin),
      _input1Pin(input1Pin),
      _input2Pin(input2Pin),
      _direction(HBRIDGE_STOPPED) {}

void HBridgeMotor::init() {
    pinMode(_input1Pin, OUTPUT);
    pinMode(_input2Pin, OUTPUT);
    _enable.init();
    stop();
}

void HBridgeMotor::setForward(float dutyPercent) {
    setDirectionPins(HBRIDGE_FORWARD);
    _enable.setDuty(dutyPercent);
}

```

```

}

void HBridgeMotor::setReverse(float dutyPercent) {
    setDirectionPins(HBRIDGE_REVERSE);
    _enable.setDuty(dutyPercent);
}

void HBridgeMotor::stop() {
    setDirectionPins(HBRIDGE_STOPPED);
    _enable.setDuty(0.0f);
}

float HBridgeMotor::getDuty() const {
    return _enable.getDuty();
}

uint8_t HBridgeMotor::getRawPwm() const {
    return _enable.getRawPwm();
}

HBridgeDirection HBridgeMotor::getDirection() const {
    return _direction;
}

void HBridgeMotor::setDirectionPins(HBridgeDirection direction) {
    _direction = direction;

    switch (direction) {
        case HBRIDGE_FORWARD:
            digitalWrite(_input1Pin, HIGH);
            digitalWrite(_input2Pin, LOW);
            break;
        case HBRIDGE_REVERSE:
            digitalWrite(_input1Pin, LOW);
            digitalWrite(_input2Pin, HIGH);
            break;
        case HBRIDGE_STOPPED:
        default:
            digitalWrite(_input1Pin, LOW);
            digitalWrite(_input2Pin, LOW);
            break;
    }
}

```

Listing 7.16 – PWM actuator interface

```

/**
 * @file PwmActuator.h
 * @brief PWM-Based Analog Actuator Driver Interface

```



```

*
* Provides a reusable abstraction for controlling an analog actuator
* via PWM output. Accepts a duty cycle percentage (0-100%) and maps
* it to 0-255 for analogWrite().
*
* Usage:
*   PwmActuator motor(6); // PWM pin 6
*   motor.init();
*   motor.setDuty(75.0); // 75% duty cycle
*/

#ifndef PWM_ACTUATOR_H
#define PWM_ACTUATOR_H

#include <Arduino.h>

/**
 * @class PwmActuator
 * @brief Controls an analog actuator via PWM duty cycle.
 */
class PwmActuator {
public:
    /**
     * @brief Construct a new PwmActuator object.
     * @param pin PWM-capable GPIO pin.
     */
    PwmActuator(uint8_t pin);

    /** @brief Initialize the PWM pin as OUTPUT, set duty to 0. */
    void init();

    /**
     * @brief Set the duty cycle (0.0 to 100.0%).
     * @param duty Duty cycle percentage. Clamped to [0, 100].
     */
    void setDuty(float duty);

    /** @brief Get the current duty cycle percentage. */
    float getDuty() const;

    /** @brief Get the current raw PWM value (0-255). */
    uint8_t getRawPwm() const;

private:
    uint8_t _pin;
    float _duty;
};

```

```
#endif // PWM_ACTUATOR_H
```

Listing 7.17 – PWM actuator implementation

```
/**
 * @file PwmActuator.cpp
 * @brief PWM-Based Analog Actuator Driver Implementation
 */

#include "PwmActuator.h"

PwmActuator::PwmActuator(uint8_t pin)
    : _pin(pin), _duty(0.0f) {}

void PwmActuator::init() {
    pinMode(_pin, OUTPUT);
    analogWrite(_pin, 0);
    _duty = 0.0f;
}

void PwmActuator::setDuty(float duty) {
    // Clamp to valid range
    if (duty < 0.0f) duty = 0.0f;
    if (duty > 100.0f) duty = 100.0f;
    _duty = duty;
    uint8_t pwmVal = (uint8_t)(_duty * 255.0f / 100.0f);
    analogWrite(_pin, pwmVal);
}

float PwmActuator::getDuty() const {
    return _duty;
}

uint8_t PwmActuator::getRawPwm() const {
    return (uint8_t)(_duty * 255.0f / 100.0f);
}
```

Listing 7.18 – DHT sensor wrapper interface

```
/**
 * @file DhtSensor.h
 * @brief DHT temperature and humidity sensor wrapper.
 *
 * Provides a small reusable interface around the Arduino DHT library.
 * The wrapper keeps the last valid temperature and humidity values so
 * application code can distinguish sensor failures from valid samples.
 */

#ifndef DHT_SENSOR_H
```

```

#define DHT_SENSOR_H

#include <Arduino.h>
#include <DHT.h>

class DhtSensor {
public:
    /**
     * @brief Construct a DHT sensor wrapper.
     * @param dataPin GPIO pin connected to the DHT data line.
     * @param sensorType DHT11, DHT21, or DHT22 constant from DHT.h.
     */
    DhtSensor(uint8_t dataPin, uint8_t sensorType);

    /** @brief Initialize the DHT driver. */
    void init();

    /**
     * @brief Read both temperature and humidity.
     * @return true when both values are valid; false on checksum/timeout error.
     */
    bool read();

    /** @brief Get the last valid temperature in degrees Celsius. */
    float getTemperatureC() const;

    /** @brief Get the last valid relative humidity percentage. */
    float getHumidityPercent() const;

    /** @brief True if the last read produced valid values. */
    bool isValid() const;

private:
    DHT _dht;
    float _temperatureC;
    float _humidityPercent;
    bool _valid;
};

#endif // DHT_SENSOR_H

```

Listing 7.19 – DHT sensor wrapper implementation

```

/**
 * @file DhtSensor.cpp
 * @brief DHT temperature and humidity sensor wrapper implementation.
 */

#include "DhtSensor.h"

```

```

#include <math.h>

DhtSensor::DhtSensor(uint8_t dataPin, uint8_t sensorType)
    : _dht(dataPin, sensorType),
      _temperatureC(NAN),
      _humidityPercent(NAN),
      _valid(false) {}

void DhtSensor::init() {
    _dht.begin();
    _temperatureC = NAN;
    _humidityPercent = NAN;
    _valid = false;
}

bool DhtSensor::read() {
    float humidity = _dht.readHumidity();
    float temperature = _dht.readTemperature();

    if (isnan(humidity) || isnan(temperature)) {
        _valid = false;
        return false;
    }

    _humidityPercent = humidity;
    _temperatureC = temperature;
    _valid = true;
    return true;
}

float DhtSensor::getTemperatureC() const {
    return _temperatureC;
}

float DhtSensor::getHumidityPercent() const {
    return _humidityPercent;
}

bool DhtSensor::isValid() const {
    return _valid;
}

```

Listing 7.20 – Analog setpoint wrapper interface

```

/**
 * @file AnalogSetpointInput.h
 * @brief Potentiometer-based setpoint input wrapper.
 *
 * Encapsulates ADC reads and maps the raw value to a configurable

```

```

* engineering range, such as a temperature setpoint in degrees Celsius.
*/

#ifndef ANALOG_SETPOINT_INPUT_H
#define ANALOG_SETPOINT_INPUT_H

#include <Arduino.h>

class AnalogSetpointInput {
public:
    /**
     * @brief Construct a setpoint input.
     * @param adcPin Analog pin connected to the potentiometer wiper.
     * @param minValue Engineering value at ADC 0.
     * @param maxValue Engineering value at ADC max.
     * @param adcResolution ADC resolution in bits, 10 for Arduino Mega.
     */
    AnalogSetpointInput(uint8_t adcPin, float minValue, float maxValue,
                        uint8_t adcResolution = 10);

    /** @brief Configure the ADC pin. */
    void init();

    /** @brief Read raw ADC value. */
    uint16_t readRaw();

    /** @brief Read and map the current value to the configured range. */
    float readValue();

    /** @brief Update the mapped engineering range. */
    void setRange(float minValue, float maxValue);

    /** @brief Return the last raw ADC sample. */
    uint16_t getLastRaw() const;

    /** @brief Return the last mapped setpoint value. */
    float getLastValue() const;

private:
    uint8_t _adcPin;
    float _minValue;
    float _maxValue;
    uint16_t _adcMax;
    uint16_t _lastRaw;
    float _lastValue;
};

#endif // ANALOG_SETPOINT_INPUT_H

```

Listing 7.21 – Analog setpoint wrapper implementation

```
/**
 * @file AnalogSetpointInput.cpp
 * @brief Potentiometer-based setpoint input wrapper implementation.
 */

#include "AnalogSetpointInput.h"

AnalogSetpointInput::AnalogSetpointInput(uint8_t adcPin, float minValue,
                                           float maxValue, uint8_t adcResolution)
    : _adcPin(adcPin),
      _minValue(minValue),
      _maxValue(maxValue),
      _adcMax((1U << adcResolution) - 1U),
      _lastRaw(0),
      _lastValue(minValue) {}

void AnalogSetpointInput::init() {
    pinMode(_adcPin, INPUT);
    _lastRaw = 0;
    _lastValue = _minValue;
}

uint16_t AnalogSetpointInput::readRaw() {
    _lastRaw = analogRead(_adcPin);
    return _lastRaw;
}

float AnalogSetpointInput::readValue() {
    uint16_t raw = readRaw();
    float ratio = (float)raw / (float)_adcMax;
    _lastValue = _minValue + ratio * (_maxValue - _minValue);
    return _lastValue;
}

void AnalogSetpointInput::setRange(float minValue, float maxValue) {
    _minValue = minValue;
    _maxValue = maxValue;
}

uint16_t AnalogSetpointInput::getLastRaw() const {
    return _lastRaw;
}

float AnalogSetpointInput::getLastValue() const {
    return _lastValue;
}
```